

Académie Régionale de Fès - Meknès
Direction Provinciale de TAZA
Centre de Préparation au Brevet de Technicien Supérieur
Lycée Technique - Taza
Filière « Développement Digital — option Web Full Stack »

RAPPORT DE PROJET DE FIN D'ÉTUDES

PANDA

Plateforme web full-stack de mise en relation entre freelances et clients

Réalisé par :
Youness BENABBOU

Encadré par :
Mme. / M. [Encadrant pédagogique]

Année universitaire 2024 – 2025

Remerciement

Au terme de ce projet de fin d'études, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce travail.

Nous adressons nos remerciements les plus sincères à notre **encadrant pédagogique** pour sa disponibilité, ses conseils avisés et son suivi attentif tout au long de la réalisation de la plateforme **Panda**. Son accompagnement et ses orientations techniques ont été déterminants pour mener ce projet à son terme.

Nous remercions également l'ensemble du corps professoral de la filière **Développement Digital — option Web Full Stack** du Centre de Préparation au BTS du Lycée Technique de Taza, pour la qualité de la formation dispensée durant ces deux années et pour les compétences techniques et humaines qu'ils nous ont transmises.

Nos remerciements vont enfin à nos **familles** et à nos **proches**, pour leur soutien moral constant, leur patience et leurs encouragements, sans lesquels ce travail n'aurait pu aboutir. Que toutes les personnes ayant participé à ce projet trouvent ici l'expression de notre reconnaissance.

Dédicace

Nous dédions ce modeste travail à nos parents, pour leurs sacrifices et leur amour inconditionnel ; à nos enseignants, qui ont éclairé notre parcours ; et à tous ceux qui croient que la technologie peut rendre le travail plus juste, plus libre et plus accessible.

Résumé

Ce rapport présente la conception et le développement de **Panda**, une plateforme web full-stack de mise en relation entre **freelances** et **clients**, réalisée dans le cadre de notre projet de fin d'études en BTS Développement Digital. L'objectif est de proposer une place de marché numérique complète couvrant l'ensemble du cycle de collaboration : publication d'offres et candidatures, vente de services packagés, contractualisation, suivi des jalons et du temps, et surtout un système de **paiement sécurisé par séquestre (escrow)** garantissant la confiance entre les parties.

Sur le plan technique, l'application repose sur une architecture moderne découplée : une **API REST Laravel 12** (PHP 8.2) exposant la logique métier, une **interface React 19** (single-page application) côté client, et une base de données **MySQL**. L'authentification s'appuie sur **Laravel Sanctum** (jetons Bearer), renforcée par l'authentification à deux facteurs (2FA) et une vérification d'identité (KYC). Le cœur financier est assuré par un **moteur de grand livre comptable** à écriture double, transactionnel et idempotent, intégré à **Stripe** pour les dépôts et les versements. La plateforme intègre par ailleurs une **messagerie temps réel** (Laravel Reverb / WebSockets), des fonctions d'**intelligence artificielle** (génération de propositions, mise en correspondance) et un module d'agences.

Les tests menés ont confirmé la fiabilité du moteur de paiement et la robustesse de l'architecture. En conclusion, ce projet illustre la mise en œuvre concrète d'une application professionnelle complète, sécurisée et évolutive, répondant à un besoin réel du marché du travail indépendant.

Mots-clés : marketplace, freelance, escrow, Laravel, React, MySQL, Sanctum, Stripe, WebSocket, API REST, paiement sécurisé, intelligence artificielle.

Abstract

This report presents the design and development of **Panda**, a full-stack web platform connecting **freelancers** with **clients**, carried out as part of our final-year project in the BTS Digital Development program. The goal is to deliver a complete digital marketplace covering the entire collaboration lifecycle: job posting and proposals, productized service selling, contracting, milestone and time tracking, and — most importantly — a secure **escrow-based payment system** that guarantees trust between parties.

Technically, the application relies on a modern decoupled architecture: a **Laravel 12 REST API** (PHP 8.2) exposing the business logic, a **React 19 single-page application** on the client side, and a **MySQL** database. Authentication is built on **Laravel Sanctum** (Bearer tokens), hardened with two-factor authentication (2FA) and identity verification (KYC). The financial core is handled by a transactional, idempotent **double-entry ledger engine** integrated with **Stripe** for deposits and payouts. The platform also features **real-time messaging** (Laravel Reverb / WebSockets), **artificial-intelligence** helpers (proposal generation, matching) and an agency module.

The tests carried out confirmed the reliability of the payment engine and the robustness of the architecture. In conclusion, this project demonstrates the concrete implementation of a complete, secure and scalable professional application that addresses a real need in the freelance labour market.

Keywords: marketplace, freelance, escrow, Laravel, React, MySQL, Sanctum, Stripe, WebSocket, REST API, secure payment, artificial intelligence.

Sommaire

Introduction générale	13
Chapitre 1 : Contexte général et conduite du projet	15
I. Cadre du projet	15
II. Le marché du travail indépendant	15
III. Problématique	16
IV. Étude de l'existant	17
1 Plateformes de référence	17
2 Tableau comparatif	18
V. Solution proposée : la plateforme Panda	18
1 Objectif principal	18
2 Objectifs spécifiques	19
3 Périmètre du projet	19
4 Modèle économique	19
5 Priorisation des fonctionnalités (MoSCoW)	20
VI. Méthodologie de conduite de projet	20
1 Principes appliqués	20
2 Découpage en sprints	21
3 Gestion des risques	21
VII. Planification — diagramme de Gantt	21
VIII. Conclusion du chapitre	22
Chapitre 2 : Analyse et spécification des besoins	23
I. Identification des acteurs	23
1 Acteurs humains	23
2 Acteurs externes (systèmes tiers)	23
II. Besoins fonctionnels	24
1 Authentification et compte	24
2 Offres, propositions et profils	24
3 Catalogue de services	24
4 Contrats et collaboration	25

5 Paiements et finance	25
6 Communication, IA et administration	25
III. Besoins non fonctionnels	26
IV. Langage de modélisation adopté	26
V. Diagramme de cas d'utilisation global	27
VI. Description textuelle des cas d'utilisation	28
1 Cas « S'authentifier »	28
2 Cas « Publier une offre »	28
3 Cas « Postuler à une offre »	28
4 Cas « Financer le séquestre et valider un jalon »	28
5 Cas « Vendre un service et passer commande »	29
6 Cas « Demander un retrait »	29
7 Cas « Vérifier l'identité (KYC) »	29
8 Cas « Gérer une agence »	30
9 Matrice de traçabilité des exigences	30
VII. Conclusion du chapitre	31
Chapitre 3 : Conception de la solution	32
I. Architecture globale	32
II. Architecture logicielle en couches	33
1 Vue en paquetages	34
III. Sécurité et authentification	35
IV. Diagramme de classes	36
V. Diagrammes de séquence dynamiques	37
1 Financement du séquestre et libération de jalon	37
2 Dépôt de fonds via Stripe	38
3 Génération de proposition par IA	39
VI. Diagrammes d'états	40
VII. Conception de la base de données	41
1 Modèle Conceptuel de Données (MCD)	41
2 Modèle Logique de Données (MLD)	41
3 Dictionnaire de données (extrait)	42

4 Règles de gestion	44
5 Vue d'ensemble des tables de la base	45
VIII. Choix technologiques	45
IX. Conclusion du chapitre	46
Chapitre 4 : Réalisation de l'application	47
I. Environnement de développement	47
II. Organisation du code source	47
1 Cartographie des principaux points d'API	48
2 Conventions et bonnes pratiques	49
3 Modèles, relations et migrations	49
III. Le cœur financier : le service de grand livre (LedgerService)	50
1 Financement du séquestre	50
2 Libération d'un jalon et répartition	51
IV. Authentification et sécurité	51
V. Communication temps réel : la messagerie	53
VI. Intégration des paiements Stripe	53
VII. Intelligence artificielle	54
VIII. Présentation des interfaces	55
1 Page d'accueil et pages publiques	55
2 Authentification et inscription	56
3 Tableaux de bord	57
4 Offres et propositions	59
5 Freelances et profils	60
6 Contrats et jalons	61
7 Paiements et portefeuille	62
8 Messagerie	63
9 Assistant IA	64
10 Design responsive et expérience utilisateur	65
IX. Conclusion du chapitre	66
Chapitre 5 : Tests, sécurité et déploiement	67
I. Stratégie de tests	67

II. Validation du moteur financier	68
III. Synthèse de la sécurité	69
IV. Déploiement	70
1 Optimisations de performance	70
2 Intégration et déploiement continu	71
V. Conclusion du chapitre	71
Chapitre 6 : Bilan et perspectives	72
I. Bilan des compétences techniques acquises	72
II. Compétences transversales	72
III. Difficultés rencontrées	73
IV. Contraintes et limites	73
V. Perspectives d'évolution	74
VI. Conclusion du chapitre	74
Conclusion générale	75
Glossaire	77
Références bibliographiques et webographie	78
Annexes	79
A Configuration de l'environnement	79
B Exemple de réponse de l'API	79
C Principaux services métier	80
D Commandes utiles	80
E Extrait du script de création des tables financières	80

Table des figures

Figure 1: Diagramme de Gantt — planification prévisionnelle du projet Panda	22
Figure 2: Diagramme de cas d'utilisation global de la plateforme Panda	27
Figure 3: Architecture générale en trois couches et services externes	32
Figure 4: Organisation en couches du back-end (MVC enrichi de services métier)	34
Figure 5: Diagramme de séquence — authentification (Sanctum + 2FA)	36
Figure 6: Diagramme de classes — entités métier principales	37
Figure 7: Diagramme de séquence — financement du séquestre et libération de jalon	38
Figure 8: Diagramme de séquence — dépôt de fonds via Stripe	39
Figure 9: Diagramme de séquence — génération de proposition assistée par IA	39
Figure 10: Diagramme d'états — cycle de vie d'un contrat	40
Figure 11: Diagramme d'états — cycle de vie d'un jalon	40
Figure 12: Modèle Conceptuel de Données (MCD) — vue principale	41
Figure 13: Page d'accueil de la plateforme Panda	55
Figure 14: Écran de connexion (e-mail/mot de passe et OAuth Google)	56
Figure 15: Écran d'inscription avec choix du rôle	57
Figure 16: Tableau de bord du client	58
Figure 17: Tableau de bord du freelance	58
Figure 18: Tableau de bord de l'administrateur	59
Figure 19: Place de marché des offres	60
Figure 20: Place de marché des freelances	61
Figure 21: Détail d'un contrat — jalons, fichiers et suivi du temps	62
Figure 22: Portefeuille et gestion des paiements	63
Figure 23: Messagerie temps réel	64
Figure 24: Assistant d'intelligence artificielle	65
Figure 25: Diagramme de déploiement — conteneurs Docker	70

Liste des tableaux

Tableau 1: Attentes croisées des clients et des freelances	16
Tableau 2: Positionnement comparatif de Panda face aux plateformes de référence	18
Tableau 3: Objectifs spécifiques du projet et bénéfices attendus	19
Tableau 4: Priorisation MoSCoW des fonctionnalités	20
Tableau 5: Découpage du projet en sprints	21
Tableau 6: Principaux risques du projet et mesures de réduction	21
Tableau 7: Acteurs humains de la plateforme Panda	23
Tableau 8: Acteurs externes intégrés à la plateforme	23
Tableau 9: Besoins non fonctionnels de la plateforme	26
Tableau 10: Matrice de traçabilité des exigences vers les composants	31
Tableau 11: Synthèse des couches de l'architecture	33
Tableau 12: Décomposition logique en paquetages	35
Tableau 13: Dictionnaire de données — table wallets	42
Tableau 14: Dictionnaire de données — table transactions	43
Tableau 15: Dictionnaire de données — table contracts	43
Tableau 16: Dictionnaire de données — table milestones	44
Tableau 17: Dictionnaire de données — tables job_postings et proposals	44
Tableau 18: Vue d'ensemble des tables de la base, par domaine	45
Tableau 19: Pile technologique back-end	46
Tableau 20: Pile technologique front-end	46
Tableau 21: Environnement et outils de développement	47
Tableau 22: Extrait de l'API — authentification, offres et catalogue	48
Tableau 23: Extrait de l'API — contrats et jalons	48
Tableau 24: Extrait de l'API — paiements, IA et administration	49
Tableau 25: Services d'intelligence artificielle de la plateforme	55
Tableau 26: Couverture des tests par domaine fonctionnel	68
Tableau 27: Échantillon des cas de test du moteur financier	69
Tableau 28: Synthèse des mesures de sécurité (défense en profondeur)	69

Tableau 29: Glossaire des principaux termes	77
Tableau 30: Principaux services métier de la plateforme	80

Introduction générale

La transformation numérique a profondément bouleversé le monde du travail au cours de la dernière décennie. L'essor du **travail indépendant** — le *freelancing* — en constitue l'une des manifestations les plus marquantes : de plus en plus de professionnels choisissent de proposer leurs compétences à distance, tandis que les entreprises, de la *start-up* à la grande organisation, externalisent une part croissante de leurs projets vers des talents qualifiés, où qu'ils se trouvent. Cette nouvelle économie, dite *économie des plateformes*, repose sur des places de marché numériques capables de mettre en relation l'offre et la demande de compétences, tout en sécurisant la transaction qui les unit.

Cependant, cette mise en relation soulève une difficulté centrale : la **confiance**. Comment un client peut-il être certain de ne payer que pour un travail effectivement livré et conforme ? Comment un freelance peut-il être assuré d'être rémunéré une fois la prestation réalisée ? Sur un marché où les deux parties ne se connaissent pas et ne se rencontrent jamais physiquement, l'absence de tiers de confiance constitue un frein majeur. Les plateformes internationales de référence (Upwork, Fiverr, Malt, Freelancer.com) ont répondu à ce problème par des mécanismes de **séquestre de fonds** (escrow), de jalons de paiement et de gestion des litiges, devenus aujourd'hui des standards incontournables du secteur.

C'est dans ce contexte que s'inscrit notre projet de fin d'études : la conception et le développement de **Panda**, une plateforme web complète de mise en relation entre freelances et clients. Loin de se limiter à un simple annuaire ou à un formulaire de contact, Panda ambitionne de couvrir l'intégralité du cycle de vie d'une collaboration : depuis la découverte d'un talent ou d'une offre, jusqu'au versement final des fonds, en passant par la contractualisation, le suivi des livrables et la communication. Le projet met un soin particulier à reproduire, dans un cadre académique, les briques techniques les plus exigeantes d'une telle plateforme — au premier rang desquelles un **moteur de paiement par séquestre fiable et auditable**.

Ce travail nous a permis de mobiliser et d'approfondir l'ensemble des compétences acquises durant notre formation : modélisation et conception logicielle (UML, Merise), développement back-end avec le framework **Laravel**, développement front-end avec la bibliothèque **React**, conception de bases de données relationnelles avec **MySQL**, sécurité

applicative, intégration de services tiers (Stripe, OAuth, intelligence artificielle) et communication temps réel. Il nous a également confrontés aux réalités d'un projet d'envergure : arbitrages de périmètre, gestion de la complexité et exigence de qualité.

Le présent rapport rend compte de la démarche suivie et des résultats obtenus. Il s'organise en **six chapitres**. Le **premier** présente le contexte général du projet, la problématique, l'étude de l'existant et la méthodologie de conduite adoptée. Le **deuxième** est consacré à l'analyse et à la spécification des besoins, formalisées à l'aide du langage UML. Le **troisième** détaille la conception de la solution : architecture, sécurité, modélisation objet, diagrammes dynamiques et conception de la base de données. Le **quatrième** décrit la réalisation effective, en s'appuyant sur des extraits de code significatifs et une présentation des interfaces. Le **cinquième** aborde les tests, la sécurité et le déploiement. Le **sixième**, enfin, dresse le bilan du projet, ses limites et ses perspectives d'évolution, avant une conclusion générale.

Chapitre 1 : Contexte général et conduite du projet

Ce premier chapitre pose les fondations du projet. Il en précise le cadre académique, présente le domaine métier — le marché du travail indépendant — et la problématique à laquelle Panda entend répondre. Il propose ensuite une étude des solutions existantes, définit les objectifs et le périmètre de notre plateforme, et expose enfin la méthodologie de conduite et la planification que nous avons adoptées pour mener le projet à bien.

I. Cadre du projet

Ce projet s'inscrit dans le cadre de la formation **BTS — Développement Digital, option Web Full Stack**, une filière professionnalisante de deux années post-baccalauréat dispensée au Centre de Préparation au BTS du Lycée Technique de Taza. Le projet de fin d'études (PFE) en constitue l'aboutissement : il a pour vocation de placer l'étudiant en situation réelle de conception et de développement d'une application complète, depuis l'analyse du besoin jusqu'à la livraison d'un produit fonctionnel.

Au-delà de la simple démonstration de compétences techniques, ce travail vise à développer des qualités professionnelles essentielles : l'analyse et la formalisation d'un besoin, la planification, la prise de décision en environnement contraint, l'autonomie et la rigueur. Nous avons délibérément choisi un sujet ambitieux — une place de marché transactionnelle — afin de nous confronter aux problématiques réelles d'une application professionnelle moderne : sécurité, intégrité des données financières, montée en charge et expérience utilisateur.

II. Le marché du travail indépendant

Le *freelancing* désigne l'exercice d'une activité professionnelle de manière indépendante, généralement à la prestation ou au projet, sans lien de subordination salariale. Porté par la généralisation du travail à distance, par la flexibilité recherchée tant par les professionnels que par les entreprises, et par la maturité des outils numériques, ce mode de travail connaît une croissance soutenue à l'échelle mondiale. Les domaines concernés sont nombreux : développement informatique, design graphique, rédaction et traduction, marketing digital, conseil, montage vidéo, etc.

Ce marché met en présence deux grandes catégories d'acteurs. D'un côté, les **clients** (particuliers, entrepreneurs, entreprises) qui cherchent à confier une mission précise à un professionnel compétent, dans un délai et un budget maîtrisés. De l'autre, les **freelances** qui souhaitent trouver des missions, valoriser leur expertise, gérer leur activité et être rémunérés de façon fiable. Entre les deux s'interpose le besoin d'une **plateforme de confiance**, capable d'organiser la rencontre, d'encadrer la collaboration et — point crucial — de sécuriser le paiement.

Deux grands modèles de collaboration coexistent sur ce marché. Le premier, dit « *appel d'offres* », part du **besoin du client** : celui-ci publie une mission, reçoit des candidatures (propositions) et sélectionne le freelance le plus adapté. Le second, dit « *service packagé* », part de l'**offre du freelance** : celui-ci propose des prestations standardisées à prix fixe que le client achète directement. Panda a fait le choix d'intégrer ces **deux modèles** au sein d'une même plateforme, afin de couvrir l'ensemble des situations et de maximiser les opportunités de mise en relation.

Le tableau suivant met en regard les attentes des deux profils d'utilisateurs, qui structurent directement les fonctionnalités de la plateforme.

Attentes du client	Attentes du freelance
Trouver rapidement un profil qualifié	Accéder à un flux régulier de missions
Comparer compétences, avis et tarifs	Valoriser son expertise et son portfolio
Maîtriser le budget et les délais	Être certain d'être payé pour son travail
Ne payer que pour un travail conforme	Gérer simplement son activité et ses revenus
Communiquer et suivre l'avancement	Encadrer la relation par un contrat clair

Tableau 1: Attentes croisées des clients et des freelances

III. Problématique

La mise en relation, à elle seule, ne suffit pas. Le véritable défi d'une marketplace de services réside dans la **gestion de la confiance** entre des parties qui ne se connaissent pas. Plusieurs risques se posent concrètement :

- **Risque pour le client** : payer d'avance et ne jamais recevoir le travail, ou recevoir une prestation non conforme à ce qui avait été convenu ;

- **Risque pour le freelance** : livrer un travail conséquent et ne jamais être payé, ou subir des retards de paiement répétés ;
- **Risque d'intégrité financière** : sur une plateforme manipulant l'argent de tiers, la moindre erreur de calcul, double paiement ou incohérence de solde est inacceptable ;
- **Risque de communication** : l'absence d'un canal d'échange fiable et tracé entre les parties favorise les malentendus et complique la résolution des litiges ;
- **Risque de sécurité** : usurpation d'identité, comptes frauduleux, accès non autorisé aux données personnelles et financières.

La problématique centrale de ce projet peut donc se formuler ainsi : *comment concevoir une plateforme web qui mette efficacement en relation freelances et clients, tout en garantissant la sécurité financière, l'intégrité des transactions et la confiance entre des parties distantes ?* C'est à cette question que Panda apporte une réponse, en plaçant le **séquestre de fonds (escrow)** et la traçabilité comptable au cœur de son architecture.

IV. Étude de l'existant

Avant de concevoir notre solution, nous avons analysé les principales plateformes du marché afin d'en dégager les bonnes pratiques et les standards attendus. Quatre acteurs de référence ont été étudiés.

1 Plateformes de référence

- **Upwork** : leader mondial orienté missions à l'heure ou au forfait. Propose des contrats, un suivi du temps, un séquestre, des jalons et une facturation hebdomadaire. Modèle de référence pour la partie « *offres et propositions* » ;
- **Fiverr** : centré sur des **services packagés** (« gigs ») proposés par les freelances avec des paliers de prix (basique, standard, premium). Modèle de référence pour la partie « *catalogue de services* » ;
- **Malt** : plateforme européenne premium ciblant les profils tech et créatifs, avec un fort accent sur la qualité des profils et la facturation ;
- **Freelancer.com / ComeUp** : places de marché généralistes combinant appels d'offres et services packagés.

De cette analyse, nous retenons que les fonctionnalités jugées indispensables par le marché sont : un système de profils riches, une recherche performante, un mécanisme de candidature, un **paiement sous séquestre avec jalons**, une messagerie intégrée, un système d'évaluation,

et une gestion des litiges. Panda s'inspire de ces standards en combinant les deux grands modèles — *offres/propositions* (façon Upwork) et *services packagés* (façon Fiverr) — au sein d'une même plateforme.

2 Tableau comparatif

Le tableau suivant synthétise le positionnement de Panda par rapport aux solutions existantes.

Fonctionnalité	Upwork	Fiverr	Panda
Offres + propositions (appels d'offres)	Oui	Non	Oui
Services packagés (catalogue à paliers)	Non	Oui	Oui
Païement sous séquestre (escrow)	Oui	Oui	Oui
Jalons de paiement	Oui	Limité	Oui
Suivi du temps & facturation horaire	Oui	Non	Oui
Messagerie temps réel intégrée	Oui	Oui	Oui
Assistant IA (propositions, matching)	Partiel	Partiel	Oui
Gestion d'agences	Oui	Non	Oui
Vérification d'identité (KYC) & 2FA	Oui	Oui	Oui
Code source maîtrisé (projet académique)	Non	Non	Oui

Tableau 2: Positionnement comparatif de Panda face aux plateformes de référence

Le parti pris de Panda est de **réunir en une seule plateforme** les deux grands modèles du marché (appels d'offres et services packagés) et d'en maîtriser intégralement la chaîne technique, jusqu'au moteur de paiement, dans un cadre pédagogique.

V. Solution proposée : la plateforme Panda

Panda est une plateforme web full-stack qui couvre l'ensemble du cycle de collaboration entre freelances et clients. Elle se distingue par la richesse de son périmètre fonctionnel et par la robustesse de son socle technique, en particulier financier.

1 Objectif principal

Concevoir et développer une place de marché numérique sécurisée permettant à des clients de publier des besoins et d'acheter des services, à des freelances de proposer leurs

compétences et d'être rémunérés de manière fiable, le tout encadré par un système de paiement sous séquestre garantissant l'intérêt des deux parties.

2 Objectifs spécifiques

Objectif	Bénéfice attendu
Authentification robuste (Sanctum, 2FA, OAuth)	Sécuriser l'accès et l'identité des utilisateurs
Profils riches client / freelance / agence	Valoriser les compétences et inspirer confiance
Offres, propositions et catalogue de services	Couvrir les deux modèles du marché
Contrats, jalons, suivi du temps et fichiers	Encadrer et tracer la collaboration
Moteur de paiement par séquestre (escrow)	Garantir la sécurité financière et la confiance
Messagerie temps réel	Fluidifier la communication entre les parties
Assistant IA & vérification d'identité (KYC)	Améliorer la productivité et la sécurité
Back-office d'administration & de modération	Piloter, modérer et arbitrer les litiges

Tableau 3: Objectifs spécifiques du projet et bénéfices attendus

3 Périmètre du projet

Périmètre fonctionnel. Inscription et authentification multi-rôles ; gestion des profils ; publication d'offres et soumission de propositions ; catalogue de services packagés et commandes ; contrats, jalons, fichiers, suivi du temps et extensions ; portefeuille, séquestre, retraits et facturation ; messagerie temps réel ; évaluations ; notifications ; intelligence artificielle ; agences ; vérification d'identité ; administration.

Périmètre technique. API REST Laravel 12 (PHP 8.2), SPA React 19 (Vite, TailwindCSS), base de données MySQL, authentification Sanctum, intégration Stripe, WebSockets via Laravel Reverb, et un serveur de modèle de langage (Ollama/Mistral) pour les fonctions d'IA.

Hors périmètre. Application mobile native, internationalisation multi-devises avancée, intégrations comptables externes et programmes de fidélité ne font pas partie de la version présentée, mais sont envisagés comme perspectives d'évolution (cf. chapitre 6).

4 Modèle économique

Comme les plateformes de référence, Panda repose sur un modèle de **commission**. La plateforme se rémunère à la libération des fonds, en prélevant un pourcentage sur le montant versé au freelance (commission paramétrable, fixée à 10 % par défaut). S'y ajoutent des frais

fixes sur les retraits et, optionnellement, des **abonnements** offrant des avantages (mise en avant, crédits de candidature). Ce modèle aligne l'intérêt de la plateforme sur la réussite des transactions : elle ne gagne que lorsque la collaboration aboutit. Les paramètres financiers sont centralisés dans la table `platform_settings`, modifiables par l'administrateur sans redéploiement.

5 Priorisation des fonctionnalités (MoSCoW)

Pour piloter le périmètre, nous avons priorisé les fonctionnalités selon la méthode **MoSCoW** (*Must, Should, Could, Won't*), qui distingue l'essentiel du souhaitable.

Priorité	Fonctionnalités
Must (indispensable)	Authentification, offres/propositions, contrats, jalons, séquestre, libération, portefeuille, messagerie
Should (important)	Catalogue de services, retraits Stripe, évaluations, notifications, vérification d'identité, recherche
Could (souhaitable)	Assistant IA, agences, listes de talents, suivi du temps, abonnements
Won't (hors version)	Application mobile native, multi-devises, intégrations comptables externes

Tableau 4: Priorisation MoSCoW des fonctionnalités

VI. Méthodologie de conduite de projet

Compte tenu de l'ampleur du périmètre et de la nature évolutive des besoins, nous avons adopté une démarche **agile et itérative**, inspirée de Scrum mais adaptée aux contraintes d'un projet académique. Plutôt que de figer l'intégralité des spécifications en amont (approche en cascade), nous avons découpé le travail en **incréments fonctionnels** livrables et testables, organisés en *sprints* d'environ deux à trois semaines.

1 Principes appliqués

- **Développement incrémental** : chaque sprint livre un module complet et fonctionnel (authentification, puis offres, puis contrats, puis paiements, etc.) ;
- **Priorisation par la valeur et le risque** : les briques les plus risquées (le moteur de paiement) ont été traitées tôt et consolidées en priorité ;
- **Intégration continue** : le code est versionné avec **Git**, chaque fonctionnalité étant développée puis intégrée à la branche principale après validation ;
- **Tests au fil de l'eau** : les comportements critiques (séquestre, libération de jalon) sont couverts par des tests automatisés dès leur écriture ;

- **Revue régulière** : un point d'avancement périodique permet de réajuster les priorités et de valider les incréments.

2 Découpage en sprints

Sprint	Objectif principal	Durée
S0 — Cadrage	Analyse des besoins, conception UML, maquettes, mise en place du dépôt	2 sem.
S1 — Socle	Authentification, modèles, migrations, profils, sécurité	3 sem.
S2 — Marketplace	Offres, propositions, recherche, catalogue de services	3 sem.
S3 — Collaboration	Contrats, jalons, fichiers, suivi du temps, extensions	3 sem.
S4 — Paiements	Portefeuille, séquestre, LedgerService, Stripe, retraits	3 sem.
S5 — Communication & IA	Messagerie temps réel, IA, catalogue, agences	3 sem.
S6 — Qualité & livraison	Tests, sécurité, dockerisation, rapport, soutenance	3 sem.

Tableau 5: Découpage du projet en sprints

3 Gestion des risques

Tout projet comporte des risques. Les avoir identifiés en amont nous a permis d'anticiper des mesures de réduction et de protéger l'avancement.

Risque	Impact	Mesure de réduction
Complexité du moteur de paiement	Élevé	Traitement prioritaire, tests approfondis, garde-fous
Ampleur du périmètre fonctionnel	Élevé	Découpage en sprints, priorisation MoSCoW
Temps limité (parallèle aux cours)	Moyen	Planning Gantt, incréments livrables
Intégration de services tiers	Moyen	Maquettage, environnement de test (clés sandbox)
Régressions lors des évolutions	Moyen	Tests automatisés, versionnement Git

Tableau 6: Principaux risques du projet et mesures de réduction

VII. Planification — diagramme de Gantt

La planification du projet a été formalisée au moyen d'un **diagramme de Gantt**, qui ordonne les grandes phases dans le temps et met en évidence leur durée et leur enchaînement. Cette représentation nous a permis de répartir l'effort de manière équilibrée et de suivre l'avancement par rapport aux échéances. La figure ci-dessous présente le planning prévisionnel.

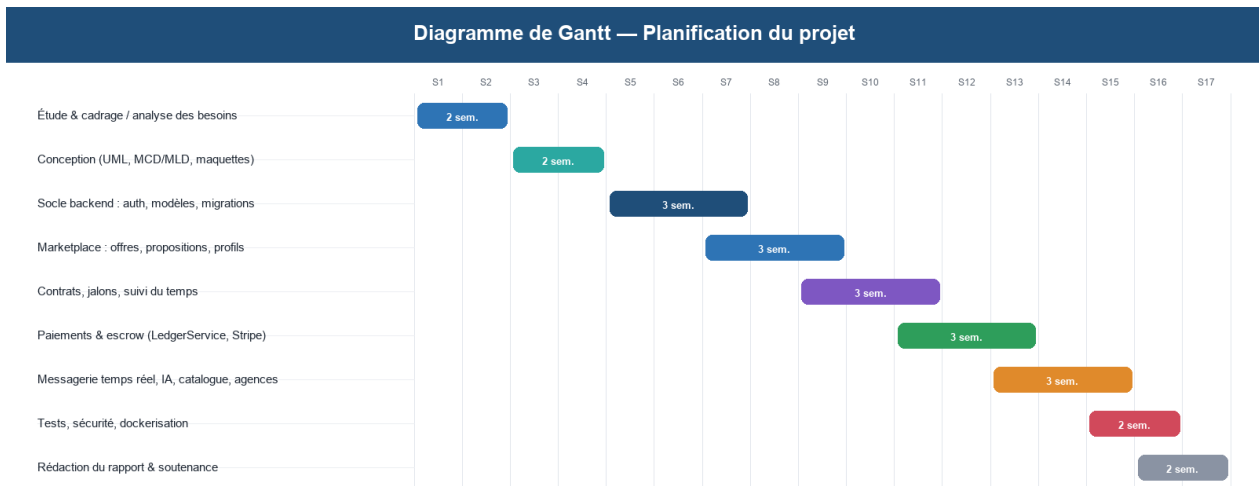


Figure 1: Diagramme de Gantt — planification prévisionnelle du projet Panda

Les phases de conception et de développement du socle ont volontairement été dotées d'une marge confortable, car elles conditionnent l'ensemble des modules suivants. La phase de paiement, identifiée comme la plus critique, a fait l'objet d'une attention particulière et de tests approfondis avant de poursuivre.

VIII. Conclusion du chapitre

Ce premier chapitre a défini le cadre du projet, caractérisé le marché du travail indépendant et formulé la problématique de la confiance, au cœur de toute place de marché de services. L'étude de l'existant a confirmé les standards attendus et précisé le positionnement de Panda, qui réunit appels d'offres et services packagés autour d'un moteur de paiement sécurisé. Enfin, la méthodologie agile et la planification adoptées ont été exposées. Le chapitre suivant approfondit **l'analyse et la spécification des besoins**, formalisées à l'aide du langage UML.

Chapitre 2 : Analyse et spécification des besoins

L'analyse des besoins est une étape déterminante qui conditionne la qualité de la conception et de la réalisation. Ce chapitre identifie les **acteurs** du système, recense les **besoins fonctionnels** et **non fonctionnels**, puis les formalise à l'aide du langage **UML**. Il présente le diagramme de cas d'utilisation global, complété par la description textuelle des cas d'utilisation les plus significatifs.

I. Identification des acteurs

Un acteur représente une entité externe — humaine ou logicielle — qui interagit avec le système. Panda distingue les acteurs humains, porteurs de rôles métier, des acteurs externes que sont les services tiers intégrés.

1 Acteurs humains

Acteur	Description et responsabilités
Visiteur	Internaute non authentifié : parcourt les offres, les freelances et le catalogue, consulte les pages publiques, puis s'inscrit ou se connecte.
Client	Utilisateur authentifié qui publie des offres, étudie les propositions reçues, achète des services, finance le séquestre, valide les jalons et évalue les freelances.
Freelance	Utilisateur authentifié qui complète son profil et son portfolio, postule aux offres, vend des services packagés, livre les jalons, suit son temps et demande ses retraits.
Agence	Entité regroupant plusieurs freelances sous une même bannière ; le propriétaire invite des membres et gère l'activité collective.
Administrateur	Supervise la plateforme : modération du catalogue, gestion des utilisateurs, validation des vérifications d'identité, traitement des retraits et arbitrage des litiges.

Tableau 7: Acteurs humains de la plateforme Panda

2 Acteurs externes (systèmes tiers)

Acteur externe	Rôle dans le système
Stripe	Encaissement des dépôts (Checkout) et versement des retraits (Connect/Payout).
Google (OAuth)	Fournisseur d'identité pour l'authentification sociale via Socialite.
Serveur IA (Ollama / Mistral)	Génération de propositions, mise en correspondance et analyse de profils.
Service de notifications Web Push	Acheminement des notifications navigateur (VAPID).

Tableau 8: Acteurs externes intégrés à la plateforme

II. Besoins fonctionnels

Les besoins fonctionnels décrivent les services que le système doit rendre. Compte tenu de la richesse du périmètre, ils sont présentés par **module fonctionnel**.

1 Authentification et compte

La gestion du compte constitue le point d'entrée de la plateforme et le socle de la confiance. Elle doit être à la fois simple pour l'utilisateur et solide face aux menaces.

- Inscription par e-mail/mot de passe ou via Google (OAuth) ☐
- Connexion sécurisée par jeton et déconnexion ☐
- Vérification de l'adresse e-mail et du numéro de téléphone ☐
- Authentification à deux facteurs (2FA) par code temporel (TOTP) et codes de secours ☐
- Réinitialisation du mot de passe et gestion du profil (avatar, informations) ☐
- Vérification d'identité (KYC) avec pièces justificatives.

2 Offres, propositions et profils

Ce module incarne le premier modèle de mise en relation, à l'initiative du **client** qui exprime un besoin auquel les freelances répondent par des propositions.

- Publication, modification et suppression d'offres par le client ☐
- Recherche et filtrage des offres et des freelances (catégorie, budget, compétences) ☐
- Soumission d'une proposition par le freelance (lettre, montant) ☐
- Acceptation, refus ou retrait d'une proposition ☐
- Gestion du profil freelance : titre, biographie, compétences, portfolio, disponibilité ☐
- Enregistrement de freelances favoris et constitution de listes de talents.

3 Catalogue de services

Le catalogue incarne le second modèle, à l'initiative du **freelance** qui propose des prestations standardisées à prix fixe, que le client achète directement.

- Création par le freelance de services packagés à plusieurs paliers (basique, standard, premium) ☐
- Modération du catalogue par l'administrateur (approbation/rejet) ☐
- Achat d'un palier par le client (commande) ☐

- Livraison, validation et évaluation de la commande.

4 Contrats et collaboration

Une fois l'accord conclu, le **contrat** encadre et trace la collaboration, du premier livrable jusqu'à la clôture, en structurant le travail en jalons mesurables.

- Création automatique d'un contrat à l'acceptation d'une proposition ou d'une commande ☐
- Découpage du contrat en **jalons** (montant, échéance) ☐
- Soumission, approbation ou rejet d'un jalon ☐
- Partage de **fichiers** versionnés liés au contrat ☐
- **Suivi du temps** pour les contrats horaires (démarrage/arrêt, récapitulatif hebdomadaire) ☐
- Demande et réponse d'**extension** de délai ☐
- Suivi d'activité, analyse et génération de **PDF** du contrat.

5 Paiements et finance

Le module financier est le plus **sensible** : il manipule l'argent réel des utilisateurs et matérialise la promesse de confiance de la plateforme à travers le séquestre.

- Portefeuille personnel avec solde disponible, en séquestre et en attente ☐
- Dépôt de fonds via Stripe ☐
- **Financement du séquestre** d'un contrat par le client ☐
- **Libération d'un jalon** : versement au freelance après prélèvement de la commission ☐
- Demande de **retrait** par le freelance et validation/refus par l'administrateur ☐
- Configuration Stripe Connect pour les versements ☐
- Facturation hebdomadaire pour les contrats horaires et abonnements.

6 Communication, IA et administration

Ces modules transverses fluidifient les échanges entre les parties, accélèrent les tâches répétitives grâce à l'IA et outillent la supervision de la plateforme.

- **Messagerie temps réel** : conversations, pièces jointes, accusés de lecture, réactions ☐
- **Notifications** in-app et Web Push ☐

- **Assistant IA** : génération de propositions, mise en correspondance, analyse de profil, recherche intelligente ☐
- **Gestion d'agences** : invitations, membres, transfert de propriété ☐
- **Back-office** : tableau de bord, gestion des utilisateurs, modération, KYC, finance, litiges.

III. Besoins non fonctionnels

Les besoins non fonctionnels caractérisent la **qualité** attendue du système, indépendamment des fonctionnalités. Ils sont au moins aussi importants que les premiers pour une plateforme transactionnelle.

Catégorie	Exigence
Sécurité	Mots de passe hachés (bcrypt), jetons Sanctum, 2FA, vérification d'identité, limitation de débit (throttling), politiques d'autorisation, journal d'audit, vérification de signature des webhooks.
Intégrité financière	Atomicité (transactions SQL), verrouillage (FOR UPDATE), écriture comptable double, idempotence des opérations, précision décimale.
Performance	Réponses rapides de l'API, requêtes Eloquent optimisées, index plein-texte pour la recherche, cache Redis, pagination.
Fiabilité	Comportement stable face aux entrées invalides, gestion explicite des erreurs, cohérence garantie des soldes.
Ergonomie	Interface responsive (TailwindCSS), navigation intuitive, retours visuels (toasts), temps réel pour la messagerie.
Maintenabilité	Architecture en couches, séparation des responsabilités, services métier réutilisables, code versionné et conteneurisé.
Scalabilité	Découplage front/back, file d'attente asynchrone, conception sans état de l'API, déploiement par conteneurs.

Tableau 9: Besoins non fonctionnels de la plateforme

IV. Langage de modélisation adopté

Pour formaliser l'analyse et la conception, nous avons adopté le langage **UML** (*Unified Modeling Language*), standard de la modélisation des systèmes logiciels orientés objet. UML offre une famille de diagrammes complémentaires permettant de représenter aussi bien la structure statique du système (diagrammes de cas d'utilisation, de classes) que son comportement dynamique (diagrammes de séquence, d'états). Pour la base de données, nous avons complété UML par la méthode **Merise** (modèles conceptuel et logique de données),

bien adaptée à la conception relationnelle.

Dans ce chapitre, nous nous concentrons sur le **diagramme de cas d'utilisation**, qui offre une vision globale et fonctionnelle du système, intelligible par toutes les parties prenantes. Les diagrammes structurels et dynamiques sont présentés au chapitre 3, consacré à la conception.

V. Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation global synthétise les interactions entre les acteurs et les grandes fonctionnalités du système. Chaque *cas d'utilisation* représente une unité de service rendue à un acteur. La figure suivante présente cette vue d'ensemble.

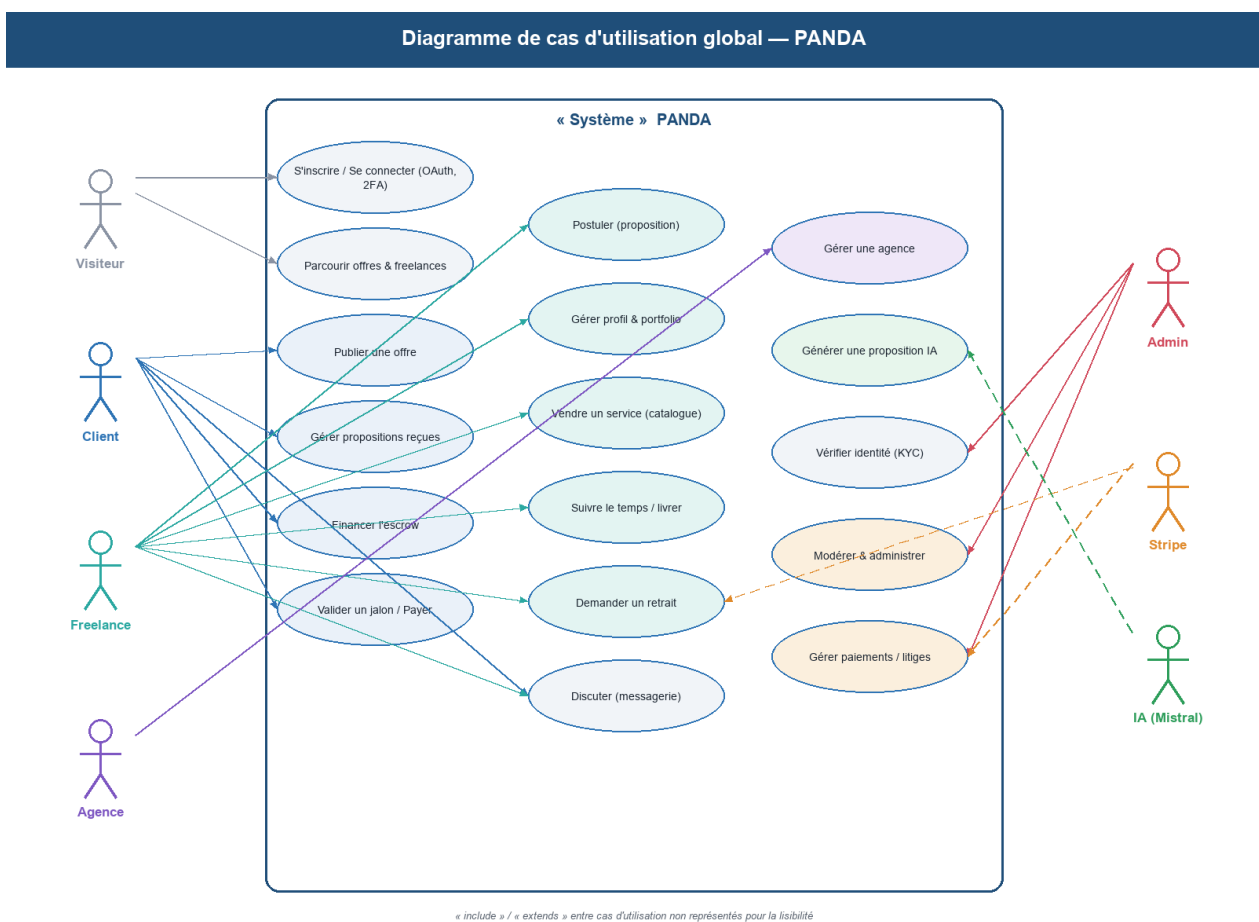


Figure 2: Diagramme de cas d'utilisation global de la plateforme Panda

On observe que de nombreux cas dépendent du cas « *S'authentifier* » (relation d'inclusion) et que les acteurs externes (Stripe, IA) interviennent en support de cas spécifiques. Les rôles Client et Freelance partagent certains cas transverses (messagerie, gestion du profil), tandis que l'Administrateur dispose de cas de supervision exclusifs.

VI. Description textuelle des cas d'utilisation

Afin de préciser le comportement attendu, nous détaillons ci-après quelques cas d'utilisation représentatifs sous forme de fiches descriptives.

1 Cas « S'authentifier »

Acteur(s)	Visiteur
Pré-conditions	Le visiteur possède un compte (ou en crée un).
Scénario nominal	1) Le visiteur saisit son e-mail et son mot de passe. 2) Le système applique une limitation de débit, vérifie les identifiants (bcrypt). 3) Si la 2FA est active, un code temporel est exigé. 4) Le système émet un jeton Bearer (Sanctum). 5) Le client stocke le jeton et accède à son espace.
Scénarios alternatifs	2a) Identifiants invalides → message d'erreur. 3a) Code 2FA erroné → nouvel essai. 1b) Authentification via Google (OAuth) en alternative.
Post-conditions	L'utilisateur est authentifié ; un jeton est associé à sa session.

2 Cas « Publier une offre »

Acteur(s)	Client
Pré-conditions	Le client est authentifié.
Scénario nominal	1) Le client ouvre le formulaire de publication. 2) Il renseigne titre, description, catégorie, budget et type. 3) Le système valide les données (FormRequest). 4) L'offre est enregistrée et publiée. 5) Elle devient visible et indexée pour la recherche.
Scénarios alternatifs	3a) Données incomplètes → erreurs de validation affichées. 4a) Le client enregistre un brouillon avant publication.
Post-conditions	Une nouvelle offre est créée et ouverte aux propositions des freelances.

3 Cas « Postuler à une offre »

Acteur(s)	Freelance
Pré-conditions	Le freelance est authentifié et l'offre est ouverte.
Scénario nominal	1) Le freelance consulte le détail de l'offre. 2) Il rédige une proposition (lettre, montant), éventuellement assistée par l'IA. 3) Le système enregistre la proposition. 4) Le client en est notifié.
Scénarios alternatifs	2a) Le freelance demande une génération de proposition à l'IA puis l'édite. 3a) Le freelance retire sa proposition tant qu'elle n'est pas acceptée.
Post-conditions	Une proposition est rattachée à l'offre et visible par le client.

4 Cas « Financer le séquestre et valider un jalon »

Acteur(s)	Client
Pré-conditions	Un contrat actif existe et le client dispose d'un solde suffisant.
Scénario nominal	1) Le client finance le séquestre du contrat (les fonds quittent son solde disponible et sont bloqués). 2) Le freelance soumet un jalon livré. 3) Le client approuve le jalon. 4) Le système (LedgerService) prélève la commission, crédite le freelance et marque le jalon comme payé, de manière atomique et idempotente.
Scénarios alternatifs	1a) Solde insuffisant → invitation à déposer des fonds (Stripe). 3a) Le client rejette le jalon avec un motif → le freelance re-livre. 3b) Litige → gel du contrat et arbitrage admin.
Post-conditions	Le jalon est payé ; le freelance est crédité ; chaque mouvement est tracé dans le grand livre.

5 Cas « Vendre un service et passer commande »

Acteur(s)	Freelance, Client
Pré-conditions	Le freelance possède un service publié et modéré ; le client est authentifié.
Scénario nominal	1) Le freelance crée un service à paliers (basique/standard/premium). 2) L'administrateur modère et approuve le service. 3) Le client choisit un palier et passe commande (paiement). 4) Le freelance livre la commande. 5) Le client valide et évalue la prestation.
Scénarios alternatifs	2a) Service rejeté → notification au freelance avec motif. 4a) Livraison non conforme → demande de révision par le client.
Post-conditions	La commande est complétée, le freelance crédité et une évaluation enregistrée.

6 Cas « Demander un retrait »

Acteur(s)	Freelance, Administrateur
Pré-conditions	Le freelance dispose d'un solde disponible supérieur au minimum requis.
Scénario nominal	1) Le freelance demande un retrait (montant, méthode). 2) Les fonds passent du solde disponible au solde en attente. 3) L'administrateur examine la demande. 4) En cas d'approbation, un versement Stripe est déclenché ; le webhook confirme le retrait.
Scénarios alternatifs	1a) Solde insuffisant ou inférieur au minimum → refus. 3a) Rejet par l'admin → fonds re-crédités au solde disponible. 4a) Échec Stripe → re-crédit automatique et statut « échoué ».
Post-conditions	Le retrait est complété (ou rejeté) ; chaque étape est tracée dans le grand livre.

7 Cas « Vérifier l'identité (KYC) »

Acteur(s)	Freelance/Client, Administrateur
Pré-conditions	L'utilisateur est authentifié.
Scénario nominal	1) L'utilisateur soumet ses pièces justificatives. 2) Le système enregistre la demande (statut « en attente »). 3) L'administrateur consulte la file de vérification. 4) Il approuve ou rejette la demande. 5) L'utilisateur est notifié du résultat.
Scénarios alternatifs	4a) Rejet → motif communiqué et nouvelle soumission possible.
Post-conditions	Le statut de vérification de l'utilisateur est mis à jour.

8 Cas « Gérer une agence »

Acteur(s)	Agence (propriétaire), Freelance
Pré-conditions	Le propriétaire dispose d'un compte autorisé à créer une agence.
Scénario nominal	1) Le propriétaire crée l'agence (nom, présentation). 2) Il invite des freelances par jeton. 3) Les freelances acceptent ou refusent l'invitation. 4) Le propriétaire gère les membres et peut transférer la propriété.
Scénarios alternatifs	3a) Refus de l'invitation → le freelance n'est pas ajouté. 4a) Retrait d'un membre par le propriétaire.
Post-conditions	L'agence regroupe plusieurs freelances sous une bannière commune.

Ces fiches illustrent l'importance des **scénarios alternatifs** (erreurs, litiges, abandons) : sur une plateforme transactionnelle, ils sont aussi critiques que le scénario nominal et structurent une grande partie de la logique applicative.

9 Matrice de traçabilité des exigences

Afin de garantir que chaque besoin identifié trouve une réponse dans la solution, nous avons établi une matrice de traçabilité reliant les exigences aux modules qui les satisfont.

Exigence	Module / composant	Couverture
Inscription, connexion, OAuth, 2FA	AuthController, Sanctum, Socialite, TwoFactor	Complète
Publication d'offres et propositions	JobController, ProposalController	Complète
Catalogue de services et commandes	CatalogProjectController, CatalogOrderController	Complète
Contrats, jalons, fichiers, temps	ContractController et contrôleurs associés	Complète
Paie sous séquestre	LedgerService, PaymentController	Complète
Dépôts et retraits Stripe	StripeService, StripeWebhookController	Complète
Messagerie temps réel	ChatController, événements + Reverb	Complète
Notifications et Web Push	NotificationController, PushSubscription	Complète
Assistant IA	AIController, service de modèle de langage	Complète
Agences et talents	AgencyController, TalentListController	Complète
Vérification d'identité (KYC)	IdentityVerificationController	Complète
Administration et modération	AdminController, AdminFinanceController	Complète

Tableau 10: Matrice de traçabilité des exigences vers les composants

VII. Conclusion du chapitre

Ce chapitre a recensé et structuré les besoins de la plateforme. Nous avons identifié les acteurs, détaillé les besoins fonctionnels par module et précisé les exigences non fonctionnelles — au premier rang desquelles la sécurité et l'intégrité financière. Le langage UML nous a permis de formaliser ces besoins, à travers le diagramme de cas d'utilisation global et la description des cas les plus significatifs. Ces éléments constituent le cahier des charges sur lequel s'appuie la **conception** détaillée, objet du chapitre suivant.

Chapitre 3 : Conception de la solution

Après avoir spécifié les besoins, ce chapitre détaille la **conception** de Panda. Il présente successivement l'architecture globale et logicielle, les mécanismes de sécurité, la modélisation objet (diagramme de classes), les comportements dynamiques (diagrammes de séquence et d'états), puis la conception de la base de données et les choix technologiques. L'objectif est de traduire les exigences en une architecture claire, sécurisée et maintenable.

I. Architecture globale

Panda adopte une architecture **découplée** de type *client-serveur* à trois niveaux, structurée autour d'une **API REST** et d'une **application monopage** (*Single Page Application*). Le front-end React communique avec le back-end Laravel exclusivement par des appels HTTP/JSON sécurisés, l'authentification reposant sur un **jeton Bearer** (Sanctum) transmis dans l'en-tête **Authorization**. Cette séparation nette des responsabilités améliore la maintenabilité, autorise une évolution indépendante des deux couches et prépare une éventuelle ouverture à d'autres clients (application mobile).

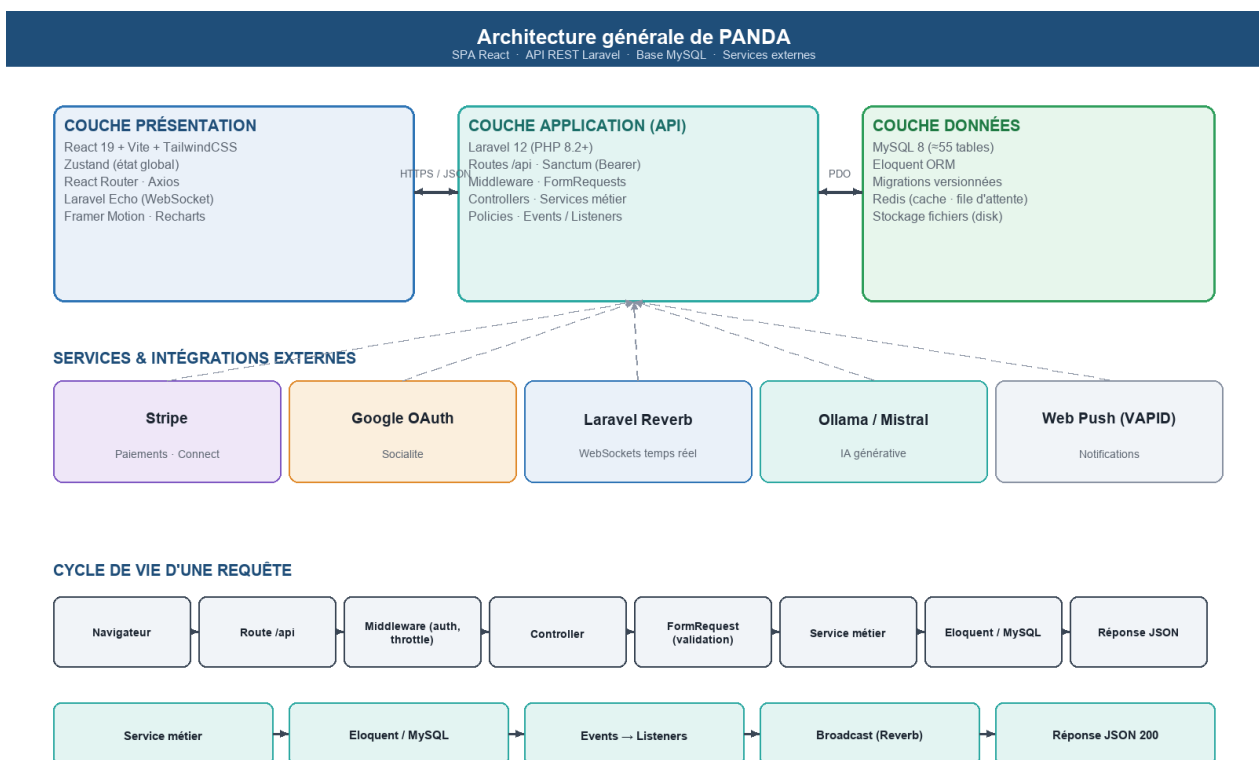


Figure 3: Architecture générale en trois couches et services externes

La **couche de présentation** (React) gère les interfaces, les formulaires, l'état applicatif (Zustand) et la communication temps réel (Laravel Echo). La **couche application** (Laravel) concentre la logique métier : routage, middlewares, validation, services et règles d'autorisation. La **couche de données** (MySQL) assure le stockage relationnel et l'intégrité référentielle, complétée par Redis pour le cache et la file d'attente. Autour de ce noyau gravitent les **services externes** : Stripe (paiements), Google (OAuth), Laravel Reverb (WebSockets) et un serveur de modèle de langage (IA).

Couche	Technologie	Rôle principal
Présentation	React 19 · Vite · TailwindCSS · Zustand	Interfaces, état, interactions, temps réel
Application	Laravel 12 (PHP 8.2) · Sanctum	API REST, logique métier, sécurité, services
Données	MySQL 8 · Eloquent ORM · Redis	Persistance, intégrité, cache, file d'attente

Tableau 11: Synthèse des couches de l'architecture

II. Architecture logicielle en couches

Côté back-end, Panda applique le patron **MVC** de Laravel, enrichi d'une **couche de services métier** dédiée. Ce choix est essentiel : il permet d'extraire la logique complexe (et notamment financière) des contrôleurs pour la concentrer dans des services réutilisables, testables et garants de la cohérence. Une requête traverse ainsi une chaîne de responsabilités clairement délimitée.

Organisation en couches du backend (MVC + Services)

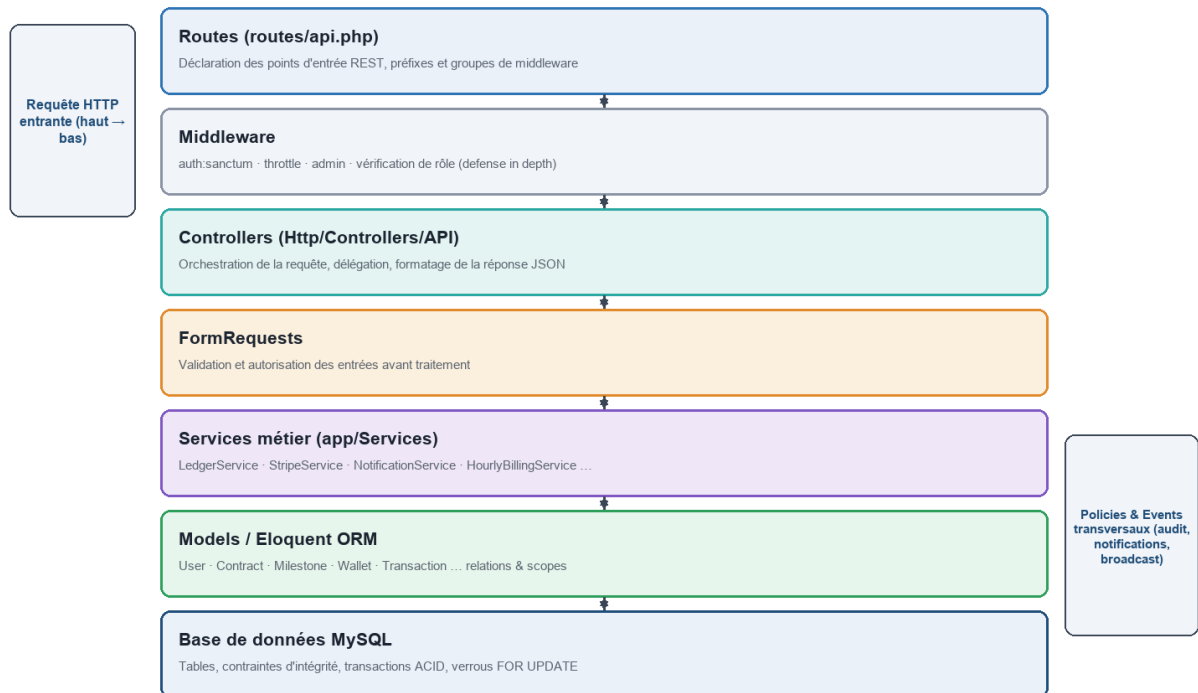


Figure 4: Organisation en couches du back-end (MVC enrichi de services métier)

Le traitement d'une requête suit le cheminement suivant : la **route** dirige vers un **contrôleur**, après passage par les **middlewares** (authentification, limitation de débit, rôle). La validation est déléguée à un **FormRequest**. Le contrôleur orchestre alors le traitement en s'appuyant sur un **service métier**, qui manipule les **modèles Eloquent** et la base de données. Des préoccupations transverses — **politiques d'autorisation, événements et écouteurs** (notifications, audit, diffusion temps réel) — s'appliquent tout au long de la chaîne. La réponse est finalement sérialisée en JSON.

Le principe directeur est la **séparation des responsabilités** : un contrôleur orchestre mais ne contient pas la logique métier sensible ; celle-ci réside dans des services (**LedgerService**, **StripeService**, **NotificationService**, etc.), seuls habilités à muter les données critiques.

1 Vue en paquetages

D'un point de vue logique, l'application se décompose en **paquetages** cohérents, chacun regroupant les contrôleurs, services et modèles d'un même domaine. Ce découpage favorise

la modularité et limite le couplage entre domaines.

Paquetage	Contenu et responsabilité
Auth	Inscription, connexion, OAuth, 2FA, réinitialisation, KYC
Marketplace	Offres, propositions, catalogue, recherche, profils
Contracts	Contrats, jalons, fichiers, temps, extensions, activité
Payments	Portefeuille, séquestre, retraits, Stripe, facturation
Communication	Messagerie temps réel, notifications, Web Push
AI	Génération, correspondance, analyse, recherche intelligente
Talent & Agency	Favoris, listes de talents, agences et invitations
Admin	Tableau de bord, modération, finance, gestion des litiges

Tableau 12: Décomposition logique en paquetages

III. Sécurité et authentification

La sécurité étant une exigence de premier ordre, Panda met en œuvre une **défense en profondeur** combinant plusieurs mécanismes complémentaires.

- **Authentification par jeton** : Laravel Sanctum émet un jeton Bearer à la connexion ; chaque requête protégée le présente dans l'en-tête **Authorization** ;
- **Authentification à deux facteurs (2FA)** : code temporel TOTP et codes de secours, pour renforcer la protection des comptes sensibles ;
- **Authentification sociale** : connexion via Google (OAuth 2.0) à l'aide de Laravel Socialite ;
- **Hachage des mots de passe** : algorithme bcrypt ; aucun mot de passe n'est stocké en clair ;
- **Limitation de débit (throttling)** : les routes d'authentification (10 req/min), de réinitialisation (5 req/min) et d'IA (20 req/min) sont protégées contre les abus ;
- **Politiques d'autorisation (Policies)** : chaque action sensible vérifie que l'utilisateur y est habilité (par exemple, seul le client d'un contrat peut en libérer les jalons) ;
- **Vérification d'identité (KYC)** : soumission et validation de pièces justificatives ;
- **Journal d'audit (audit logs)** : traçabilité des actions sensibles ;
- **Vérification de signature** des webhooks Stripe et **idempotence** des événements reçus.

Le diagramme de séquence ci-dessous illustre le processus d'authentification, intégrant la limitation de débit, la vérification des identifiants et la 2FA.

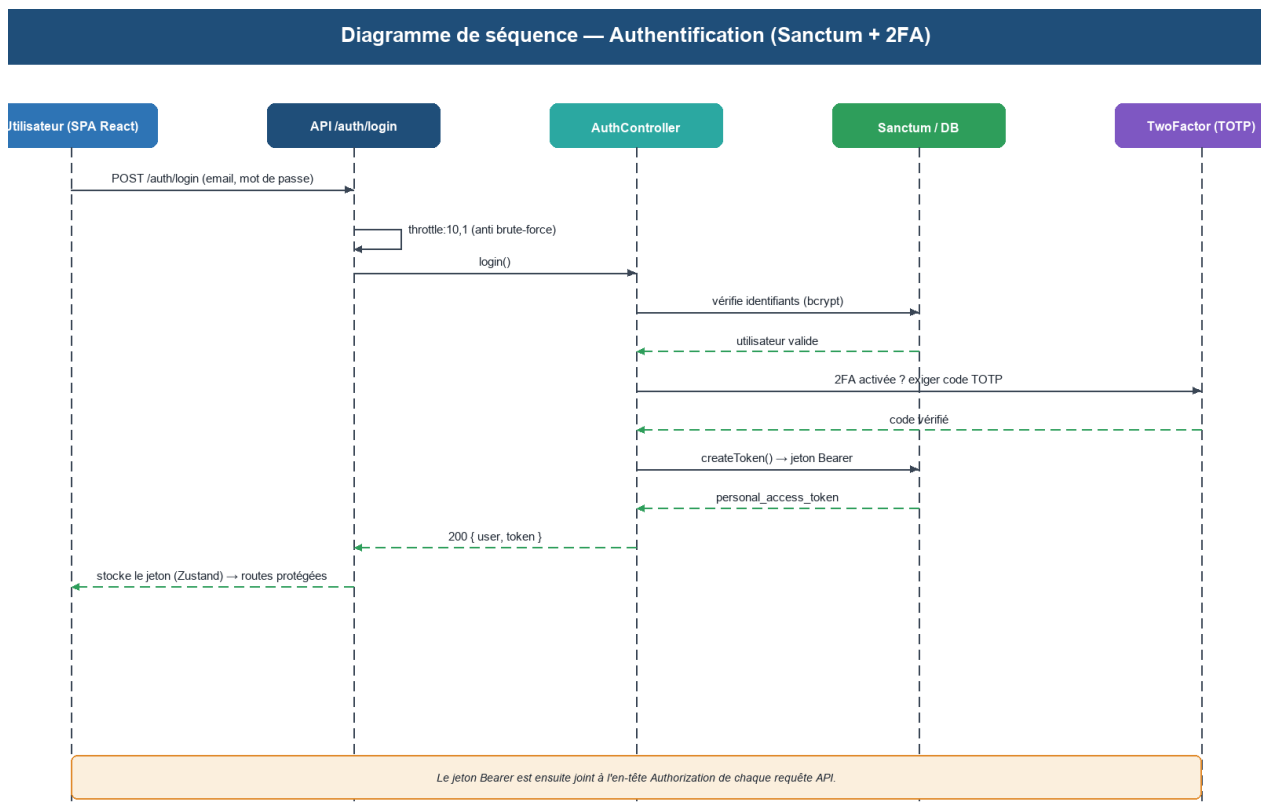


Figure 5: Diagramme de séquence — authentification (Sanctum + 2FA)

Le **cycle de vie du jeton** est maîtrisé : émis à la connexion, il peut être révoqué à la déconnexion et n'est jamais exposé au-delà du strict nécessaire. Côté client, il est conservé de façon contrôlée et systématiquement joint aux requêtes par l'intercepteur Axios. Cette approche, sans état côté serveur, est parfaitement adaptée à une application monopage et facilite la montée en charge.

La **protection des données personnelles** complète ce dispositif : les informations sensibles (pièces d'identité du KYC) sont stockées sur un disque privé, non accessible publiquement, et ne sont consultables que par l'administrateur via des points d'accès protégés. Les actions critiques sont par ailleurs consignées dans un **journal d'audit**, assurant la traçabilité indispensable à toute investigation ultérieure.

IV. Diagramme de classes

Le diagramme de classes modélise la structure statique du système : les entités métier, leurs attributs et leurs relations. Il constitue la traduction objet du domaine et la base de la conception de la base de données. La figure suivante en présente les classes principales.

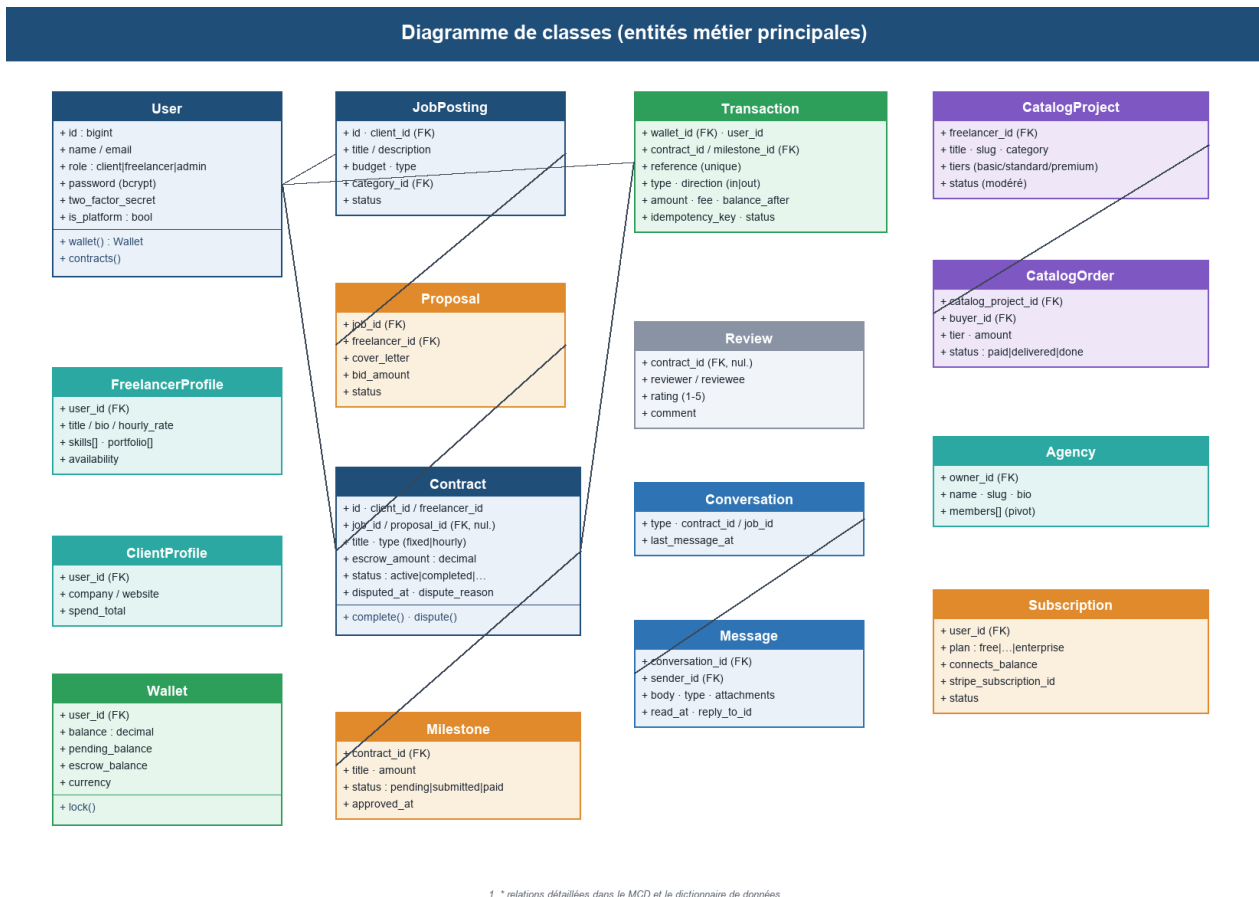


Figure 6: Diagramme de classes — entités métier principales

On distingue plusieurs regroupements cohérents : les entités d'identité (**User**, **FreelancerProfile**, **ClientProfile**, **Agency**) ; le flux de mise en relation (**JobPosting** → **Proposal** → **Contract** → **Milestone**) ; le noyau financier (**Wallet**, **Transaction**) ; la communication (**Conversation**, **Message**, **Review**) ; et le catalogue (**CatalogProject**, **CatalogOrder**). La classe **Contract** joue un rôle pivot : elle relie un client et un freelance, agrège des jalons et constitue le contexte des mouvements financiers et des conversations.

V. Diagrammes de séquence dynamiques

Au-delà de la structure, les diagrammes de séquence décrivent les **interactions temporelles** entre objets pour les scénarios les plus critiques. Nous en présentons trois.

1 Financement du séquestre et libération de jalon

Ce scénario est le cœur transactionnel de la plateforme. Il met en jeu le **LedgerService**, qui garantit l'atomicité et l'intégrité de chaque mouvement de fonds.

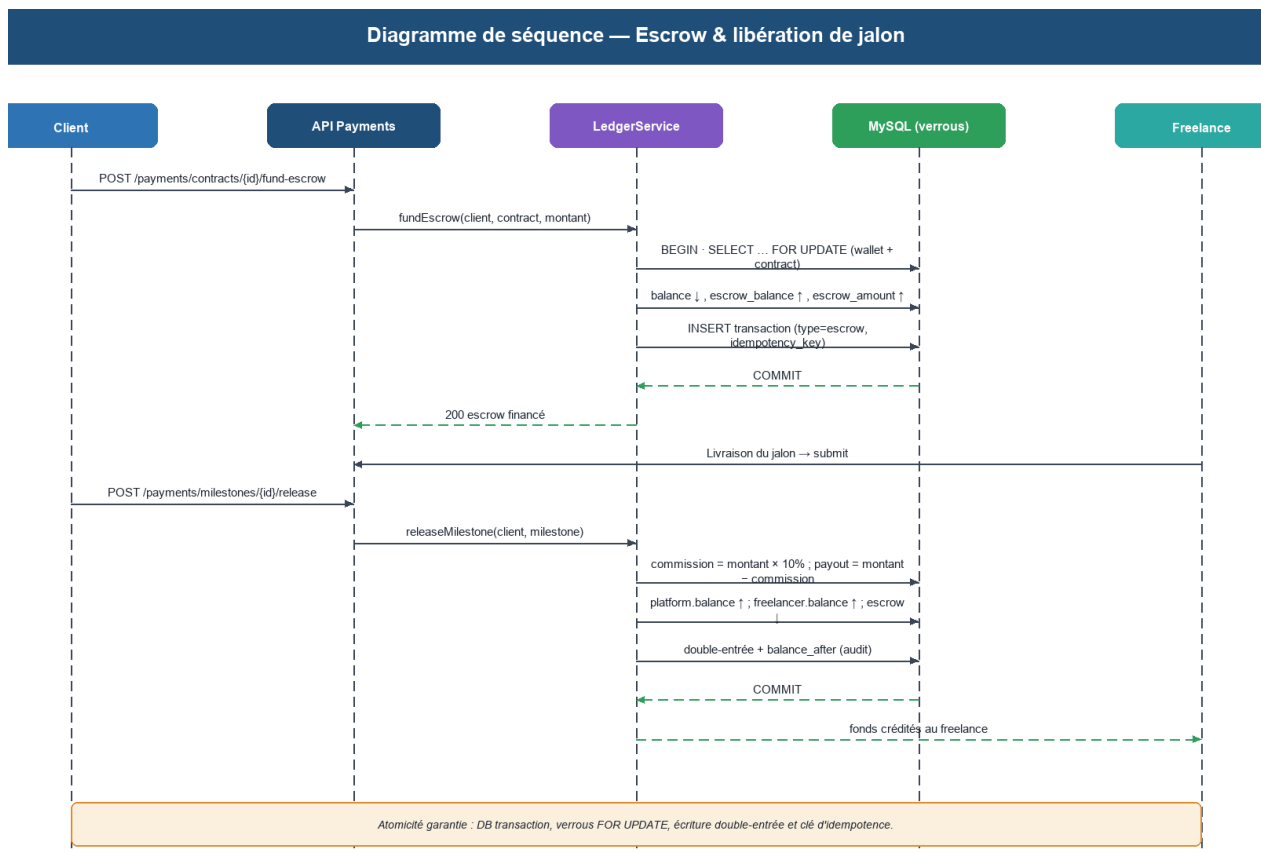


Figure 7: Diagramme de séquence — financement du séquestre et libération de jalon

À la libération, le service calcule la **commission** de la plateforme, crédite le freelance du montant net, débite le séquestre et enregistre chaque écriture en **double-entrée** avec son solde résultant (**balance_after**), le tout dans une **transaction verrouillée** et **idempotente**.

2 Dépôt de fonds via Stripe

Le dépôt s'appuie sur Stripe Checkout. La confirmation du paiement parvient de façon asynchrone par un **webhook** signé, dont l'idempotence est assurée par la table **stripe_webhook_events**.

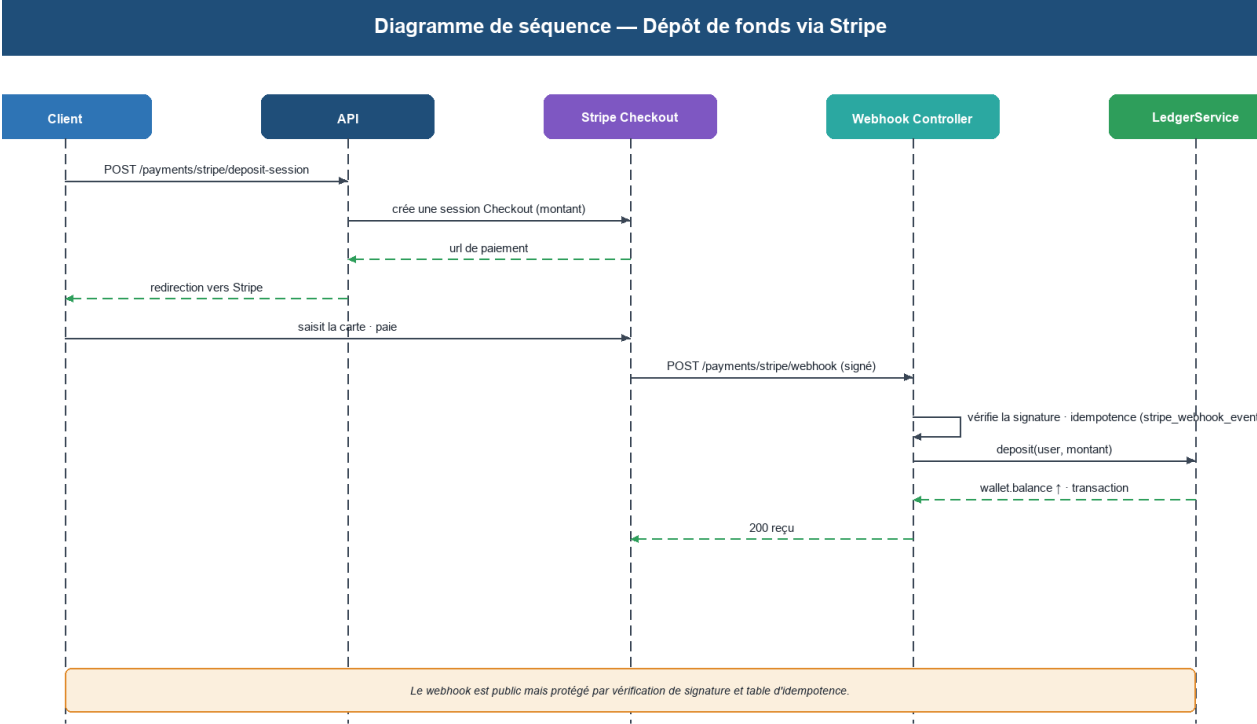


Figure 8: Diagramme de séquence — dépôt de fonds via Stripe

3 Génération de proposition par IA

Le freelance peut solliciter l'assistant IA pour rédiger une première version de proposition, qu'il édite ensuite. L'appel au modèle est limité en débit pour maîtriser les coûts, et chaque interaction est historisée.

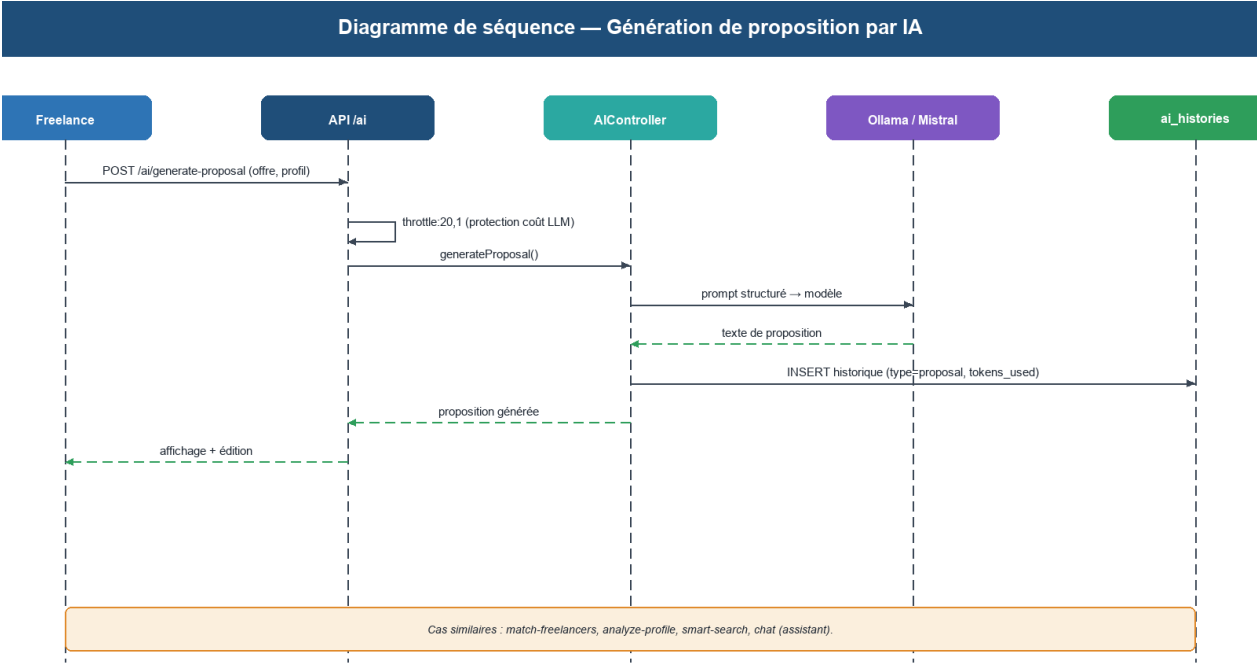


Figure 9: Diagramme de séquence — génération de proposition assistée par IA

VI. Diagrammes d'états

Les diagrammes d'états-transitions décrivent le **cycle de vie** des objets métier soumis à des changements d'état. Deux entités le justifient particulièrement : le contrat et le jalon.

Diagramme d'états — Contrat

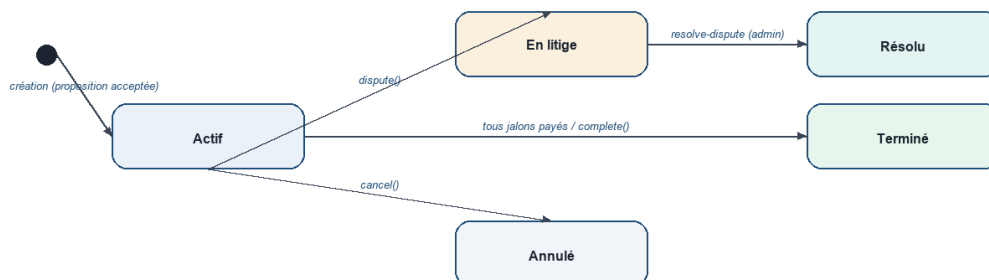


Figure 10: Diagramme d'états — cycle de vie d'un contrat

Un contrat naît **actif** à l'acceptation d'une proposition. Il peut être mené à son terme (**terminé**) lorsque tous les jalons sont payés, **annulé**, ou basculer en **litige** ; dans ce dernier cas, l'arbitrage de l'administrateur le fait évoluer vers un état **résolu**. Le gel d'un contrat en litige suspend toute libération de fonds — garde-fou implémenté directement dans le moteur de paiement.

Diagramme d'états — Jalon (Milestone)

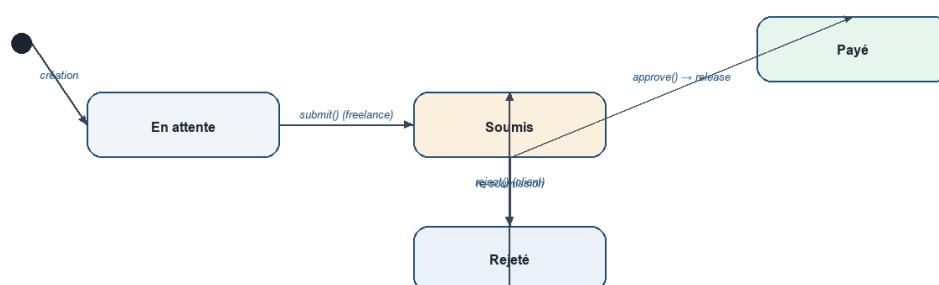


Figure 11: Diagramme d'états — cycle de vie d'un jalon

Un jalon est initialement **en attente**. Le freelance le **soumet** une fois le travail livré ; le client l'**approuve** (déclenchant le versement, état **payé**) ou le **rejette** avec un motif, autorisant une re-soumission.

VII. Conception de la base de données

La base de données relationnelle est la colonne vertébrale de la persistance. Sa conception a suivi la méthode **Merise** : un modèle conceptuel (MCD) indépendant de toute technologie, puis sa traduction en modèle logique (MLD) directement implémentable en MySQL.

1 Modèle Conceptuel de Données (MCD)

Le MCD représente les **entités**, leurs **attributs** et les **associations** qui les relient, assorties de leurs cardinalités. La figure suivante en présente une vue centrée sur les entités principales.

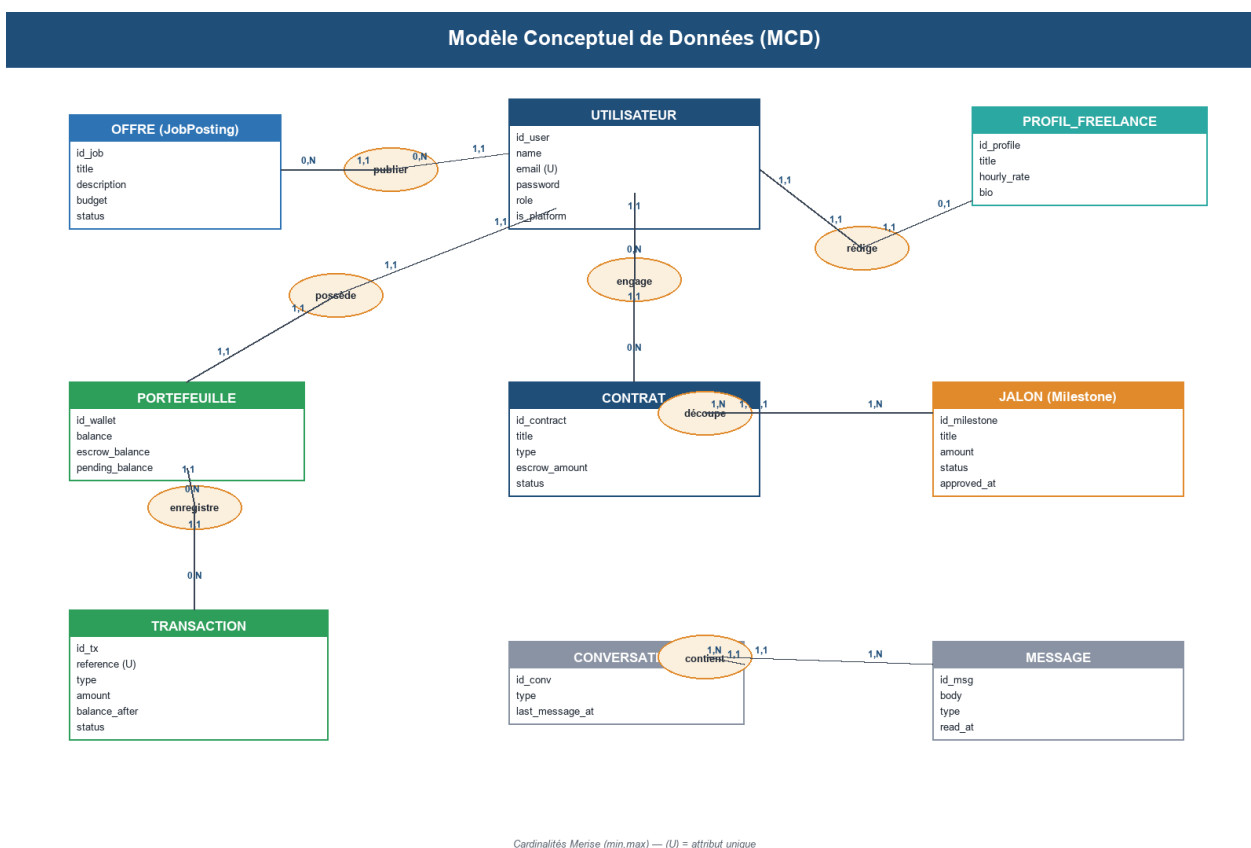


Figure 12: Modèle Conceptuel de Données (MCD) — vue principale

2 Modèle Logique de Données (MLD)

Le MLD traduit le MCD en **schéma relationnel** : chaque entité devient une table, chaque association se matérialise par des clés étrangères (notées #) ou des tables de liaison. Les clés primaires sont soulignées. Le schéma simplifié des tables principales est le suivant :

Modèle Logique de Données (extrait des tables principales)

```

users(id, name, email, password, role, is_platform, two_factor_secret, ...)
freelancer_profiles(id, #user_id, title, bio, hourly_rate, availability, ...)
client_profiles(id, #user_id, company, website, ...)
job_postings(id, #client_id, #category_id, title, description, budget, type, status)
proposals(id, #job_id, #freelancer_id, cover_letter, bid_amount, status)
contracts(id, #client_id, #freelancer_id, #job_id, #proposal_id, title, type,
          escrow_amount, status, disputed_at, dispute_reason)
milestones(id, #contract_id, title, amount, status, approved_at, #created_by)
wallets(id, #user_id, balance, pending_balance, escrow_balance, currency)
transactions(id, #wallet_id, #user_id, #contract_id, #milestone_id, reference,
             type, direction, amount, fee, balance_after, idempotency_key, status)
withdrawals(id, #user_id, #wallet_id, amount, fee, net, method, status, #reviewed_by)
conversations(id, type, #contract_id, #job_id, last_message_at)
messages(id, #conversation_id, #sender_id, body, type, attachments, read_at, #reply_to_id)
reviews(id, #contract_id, #reviewer_id, #reviewee_id, rating, comment)
catalog_projects(id, #freelancer_id, title, slug, tiers, status)
catalog_orders(id, #catalog_project_id, #buyer_id, tier, amount, status)
agencies(id, #owner_id, name, slug, bio) · subscriptions(id, #user_id, plan, status)

```

3 Dictionnaire de données (extrait)

Le dictionnaire de données précise, pour chaque table, le type et la signification des champs. Nous en donnons un extrait pour les deux tables au cœur de l'intégrité financière : **wallets** et **transactions**.

Champ (wallets)	Type	Description
id	BIGINT (PK)	Identifiant du portefeuille
user_id	BIGINT (FK)	Propriétaire du portefeuille
balance	DECIMAL(12,2)	Solde disponible (retirable)
pending_balance	DECIMAL(12,2)	Fonds en attente (retrait en cours)
escrow_balance	DECIMAL(12,2)	Fonds bloqués en séquestre
currency	VARCHAR	Devise (USD par défaut)

Tableau 13: Dictionnaire de données — table wallets

Champ (transactions)	Type	Description
id	BIGINT (PK)	Identifiant de la transaction
wallet_id / user_id	BIGINT (FK)	Portefeuille et utilisateur concernés
contract_id / milestone_id	BIGINT (FK, nul.)	Contexte financier éventuel
reference	VARCHAR (unique)	Référence lisible (DEP-..., ESC-..., PAY-...)
type	ENUM	credit, debit, escrow, release, refund, withdrawal, fee
direction	ENUM	in / out (sens du mouvement)
amount / fee	DECIMAL(12,2)	Montant et frais associés
balance_after	DECIMAL(12,2)	Solde résultant (immuable, pour l'audit)
idempotency_key	VARCHAR (nul.)	Clé garantissant l'unicité de l'opération
status	ENUM	pending, completed, failed, cancelled

Tableau 14: Dictionnaire de données — table transactions

La table **transactions** constitue un **grand livre (ledger)** : chaque ligne y est immuable et conserve le solde résultant, ce qui permet de **reconstituer et de vérifier** à tout moment le solde d'un portefeuille par simple sommation — propriété fondamentale pour l'auditabilité.

Les tables **contracts** et **milestones**, qui structurent la collaboration, sont également centrales. Leur dictionnaire est présenté ci-dessous.

Champ (contracts)	Type	Description
id	BIGINT (PK)	Identifiant du contrat
client_id / freelancer_id	BIGINT (FK)	Parties au contrat
job_id / proposal_id	BIGINT (FK, nul.)	Origine du contrat (offre/proposition)
title	VARCHAR	Intitulé du contrat
type	ENUM	fixed (forfait) / hourly (horaire)
escrow_amount	DECIMAL(12,2)	Montant actuellement sous séquestre
status	ENUM	active, completed, cancelled, disputed
disputed_at / dispute_reason	DATETIME / TEXT	Informations de litige éventuel

Tableau 15: Dictionnaire de données — table contracts

Champ (milestones)	Type	Description
id	BIGINT (PK)	Identifiant du jalon
contract_id	BIGINT (FK)	Contrat de rattachement
title	VARCHAR	Description du jalon
amount	DECIMAL(12,2)	Montant associé
status	ENUM	pending, submitted, paid, rejected
approved_at	DATETIME (nul.)	Date de validation/paiement
created_by	BIGINT (FK)	Auteur de la création du jalon

Tableau 16: Dictionnaire de données — table milestones

Champ (job_postings / proposals)	Type	Description
job.title / description	VARCHAR / TEXT	Intitulé et énoncé de l'offre
job.budget / type	DECIMAL / ENUM	Budget et nature (forfait/horaire)
job.category_id	BIGINT (FK)	Catégorie de compétence
job.status	ENUM	open, closed, draft
proposal.bid_amount	DECIMAL(12,2)	Montant proposé par le freelance
proposal.cover_letter	TEXT	Lettre de motivation
proposal.status	ENUM	pending, accepted, rejected, withdrawn

Tableau 17: Dictionnaire de données — tables job_postings et proposals

4 Règles de gestion

La cohérence du système est garantie par un ensemble de **règles de gestion** explicites, implémentées dans les services et les politiques d'autorisation :

- **RG1** — Un contrat naît exclusivement de l'acceptation d'une proposition ou d'une commande ;
- **RG2** — Seul le client d'un contrat peut financer son séquestre et libérer ses jalons ;
- **RG3** — Un jalon ne peut être libéré que s'il a été *soumis* et que le séquestre est suffisant ;
- **RG4** — Un contrat *en litige* bloque toute libération de fonds jusqu'à arbitrage ;
- **RG5** — La commission de la plateforme est prélevée à la libération (10 % par défaut) ;
- **RG6** — Un retrait est soumis à un montant minimum et à un frais fixe, puis validé par un admin ;
- **RG7** — Toute opération financière est atomique, verrouillée et idempotente ;

- **RG8** — Un service du catalogue n'est visible qu'après modération par l'administrateur.

5 Vue d'ensemble des tables de la base

Le schéma complet compte une cinquantaine de tables. Le tableau suivant les regroupe par domaine fonctionnel afin d'en offrir une vision synthétique.

Domaine	Tables principales
Identité & comptes	users, freelancer_profiles, client_profiles, identity_verifications, personal_access_tokens
Offres & propositions	job_postings, proposals, saved_jobs, categories, skills, freelancer_skills
Catalogue de services	catalog_projects, catalog_project_images, catalog_orders, catalog_reviews, saved_catalog_projects
Contrats & collaboration	contracts, milestones, contract_files, contract_activities, time_logs, contract_extensions
Finance	wallets, transactions, withdrawals, weekly_invoices, platform_settings, stripe_webhook_events
Abonnements	subscriptions, subscription_items
Communication	conversations, conversation_participants, messages, message_reactions
Notifications	notifications, push_subscriptions
Talents & agences	saved_freelancers, talent_lists, talent_list_freelancers, agencies, agency_members, agency_invitations
Évaluations & IA	reviews, ai_histories
Transverses & système	audit_logs, tax_documents, portfolios, sessions, cache, jobs, failed_jobs

Tableau 18: Vue d'ensemble des tables de la base, par domaine

Cette organisation reflète fidèlement les modules fonctionnels de l'application et illustre la richesse du domaine couvert par Panda. Les contraintes d'intégrité référentielle (clés étrangères) garantissent la cohérence entre ces tables — par exemple, la suppression d'un utilisateur propage en cascade la suppression de son portefeuille et de ses profils.

VIII. Choix technologiques

Les technologies ont été sélectionnées pour leur maturité, leur écosystème et leur adéquation au besoin. Les tableaux suivants récapitulent la pile retenue.

Back-end	Rôle
Laravel 12 (PHP 8.2)	Framework MVC, API REST, écosystème riche
Laravel Sanctum	Authentification par jeton pour SPA
Laravel Socialite	Authentification OAuth (Google)
Laravel Reverb	Serveur WebSocket pour le temps réel
Stripe PHP SDK	Paielements, Checkout et Connect
MySQL 8 · Eloquent ORM	Base relationnelle et couche d'abstraction
Redis	Cache, sessions et file d'attente

Tableau 19: Pile technologique back-end

Front-end	Rôle
React 19 + Vite	Bibliothèque UI et outil de build rapide
TailwindCSS	Framework CSS utilitaire, design responsive
Zustand	Gestion d'état globale légère
React Router · Axios	Navigation SPA et client HTTP
Laravel Echo · Pusher-js	Réception des événements temps réel
Framer Motion · Recharts	Animations et visualisation de données

Tableau 20: Pile technologique front-end

IX. Conclusion du chapitre

Ce chapitre a présenté la conception complète de Panda : une architecture découplée et en couches, des mécanismes de sécurité en profondeur, une modélisation objet structurée autour du contrat, des comportements dynamiques formalisés par des diagrammes de séquence et d'états, et une base de données rigoureusement conçue selon Merise. Cette conception fournit un plan directeur cohérent. Le chapitre suivant en présente la **réalisation** concrète, illustrée par des extraits de code et la présentation des interfaces.

Chapitre 4 : Réalisation de l'application

Ce chapitre présente la mise en œuvre concrète de Panda. Après avoir décrit l'environnement de développement et l'organisation du code, il détaille — à travers des extraits significatifs — les briques techniques les plus structurantes : le moteur financier, la sécurité, la communication temps réel, l'intégration de Stripe et l'intelligence artificielle. Il s'achève par une présentation des principales interfaces utilisateur de la plateforme.

I. Environnement de développement

Le projet a été développé dans un environnement outillé et reproductible, conforme aux pratiques professionnelles.

Outil	Usage
Visual Studio Code	Éditeur principal (extensions PHP, Laravel, React)
Composer	Gestionnaire de dépendances PHP
npm / Vite	Gestion des paquets et build du front-end React
Git / GitHub	Versionnement du code et historique
MySQL · phpMyAdmin	Base de données et administration
Postman	Tests et documentation des points d'API
Docker · Docker Compose	Conteneurisation et environnement de déploiement
Ollama (Mistral)	Serveur local de modèle de langage pour l'IA

Tableau 21: Environnement et outils de développement

II. Organisation du code source

Le dépôt est organisé en deux applications distinctes — le back-end Laravel et le front-end React — favorisant le découplage. L'arborescence simplifiée est la suivante :

Arborescence simplifiée du projet

```

panda/
├── backend-laravel/
│   ├── app/
│   │   ├── Http/Controllers/API/ (Auth, Jobs, Contracts, Payments, Chat, AI, Admin...)
│   │   ├── Http/Middleware/ (auth, admin, throttle)
│   │   ├── Models/ (User, Contract, Milestone, Wallet, Transaction...)
│   │   ├── Policies/ (ContractPolicy, MilestonePolicy...)
│   │   ├── Services/ (LedgerService, StripeService, NotificationService...)
│   │   ├── Events/ · Listeners/ (MessageSent, broadcast temps réel)
│   │   └── database/migrations/ (≈ 30 migrations versionnées)
│   ├── routes/api.php (déclaration des points d'API REST)
│   └── frontend-react/
│       ├── src/
│       │   ├── pages/ (Landing, Auth, Dashboard, Jobs, Contracts, Payments...)
│       │   ├── components/ (layout, contracts, milestones, ui...)
│       │   └── store/ · api/ (état Zustand, client Axios)

```

1 Cartographie des principaux points d'API

L'API REST expose une centaine de points d'accès, organisés par domaine fonctionnel et préfixés par **/api**. Les tableaux suivants en présentent un extrait représentatif, illustrant la convention de nommage et la cohérence des verbes HTTP (GET pour lire, POST pour créer/agir, PUT pour modifier, DELETE pour supprimer).

Méthode	Point d'accès	Rôle
POST	/auth/register · /auth/login	Inscription et connexion (limités en débit)
GET	/auth/me	Profil de l'utilisateur authentifié
GET / POST	/jobs · /jobs/{job}/proposals	Offres et propositions
POST	/proposals/{p}/accept · /reject	Traitement d'une proposition
GET	/freelancers · /freelancers/{username}	Annuaire et profil public
GET / POST	/catalog · /catalog/{slug}/orders/checkout	Catalogue et commandes de services

Tableau 22: Extrait de l'API — authentification, offres et catalogue

Méthode	Point d'accès	Rôle
GET	/contracts · /contracts/{c}	Liste et détail des contrats
POST	/contracts/{c}/milestones	Création d'un jalon
POST	/milestones/{m}/submit · /approve	Soumission et validation d'un jalon
POST	/contracts/{c}/dispute · /resolve-dispute	Litige et arbitrage
GET / POST	/contracts/{c}/time/start · /stop	Suivi du temps
GET	/contracts/{c}/pdf	Génération du PDF du contrat

Tableau 23: Extrait de l'API — contrats et jalons

Méthode	Point d'accès	Rôle
GET	/payments/wallet · /overview	Portefeuille et synthèse financière
POST	/payments/contracts/{c}/fund-escrow	Financement du séquestre
POST	/payments/milestones/{m}/release	Libération d'un jalon
POST	/payments/withdrawals	Demande de retrait
POST	/payments/stripe/webhook	Webhook Stripe (signé, idempotent)
POST	/ai/generate-proposal · /match-freelancers	Services d'intelligence artificielle
GET / POST	/admin/* (dashboard, finance, kyc...)	Back-office (réservé à l'administrateur)

Tableau 24: Extrait de l'API — paiements, IA et administration

2 Conventions et bonnes pratiques

Le code respecte un ensemble de conventions garantissant lisibilité et homogénéité : nommage explicite des classes et méthodes, contrôleurs « maigres » déléguant aux services, validation centralisée dans les FormRequests, réponses JSON normalisées, et respect des standards PSR de PHP. Côté React, les composants sont organisés par domaine, l'état global est centralisé (Zustand) et les appels réseau passent par un client Axios unique muni d'intercepteurs (ajout automatique du jeton, gestion des erreurs).

3 Modèles, relations et migrations

Les **modèles Eloquent** déclarent les relations entre entités de façon expressive, ce qui simplifie considérablement les requêtes. Le modèle **Contract**, par exemple, expose ses relations avec le client, le freelance, les jalons et les transactions.

app/Models/Contract.php — relations (extrait)

```
class Contract extends Model
{
    public function client()      { return $this->belongsTo(User::class, 'client_id'); }
    public function freelancer() { return $this->belongsTo(User::class, 'freelancer_id'); }
    public function milestones() { return $this->hasMany(Milestone::class); }
    public function transactions(){ return $this->hasMany(Transaction::class); }
    public function conversation(){ return $this->hasOne(Conversation::class); }
}
```

Le schéma de la base est décrit par des **migrations** versionnées, garantissant qu'il puisse être recréé à l'identique sur n'importe quel environnement. La migration des tables financières illustre l'usage des types décimaux et des contraintes de clés étrangères.

database/migrations — table wallets (extrait)

```
Schema::create('wallets', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->decimal('balance', 12, 2)->default(0);
    $table->decimal('pending_balance', 12, 2)->default(0);
    $table->decimal('escrow_balance', 12, 2)->default(0);
    $table->string('currency')->default('USD');
    $table->timestamps();
});
```

III. Le cœur financier : le service de grand livre (LedgerService)

Le **LedgerService** constitue la **source unique de vérité** pour tout mouvement d'argent dans Panda. Aucun contrôleur n'écrit directement dans les tables **wallets** ou **transactions** : toute opération passe par ce service, qui garantit six propriétés fondamentales.

1. Toute opération s'exécute dans une **transaction SQL** (atomicité : tout ou rien) ;
2. Chaque ligne de portefeuille est **verrouillée** (**SELECT ... FOR UPDATE**) avant mutation, évitant les conditions de course ;
3. Chaque écriture est en **double-entrée** : tout débit a un crédit correspondant ;
4. Chaque transaction enregistre **balance_after** de façon **immuable** pour la réconciliation ;
5. Les **clés d'idempotence** sont honorées : un appel dupliqué renvoie le résultat initial ;
6. Un **invariant** est vérifié : la somme des mouvements d'un portefeuille doit égaler son solde.

1 Financement du séquestre

Lorsqu'un client finance le séquestre, les fonds quittent son solde disponible (**balance**) pour rejoindre son solde bloqué (**escrow_balance**) et le séquestre du contrat (**escrow_amount**). L'extrait suivant illustre la rigueur transactionnelle de l'opération.

[app/Services/LedgerService.php — fundEscrow\(\)](#) (extrait)

```

public function fundEscrow(User $client, Contract $contract, float $amount,
                          ?string $idempotencyKey = null): Transaction
{
    return DB::transaction(function () use ($client, $contract, $amount, $idempotencyKey) {
        if ($tx = $this->findByIdempotencyKey($idempotencyKey)) return $tx; // rejouable

        $wallet = $this->lockWallet($client);           // SELECT ... FOR UPDATE
        if ((float) $wallet->balance < $amount)
            throw new RuntimeException('Solde insuffisant pour le séquestre');

        $wallet->balance      = bcsub($wallet->balance, $amount, 2); // précision décimale
        $wallet->escrow_balance = bcadd($wallet->escrow_balance, $amount, 2);
        $wallet->save();
        $contract->escrow_amount = bcadd($contract->escrow_amount, $amount, 2);
        $contract->save();

        return Transaction::create([ /* type=escrow, direction=out, balance_after, ... */ ]);
    });
}

```

Le recours à **bcadd/bcsub** (arithmétique de précision arbitraire) plutôt qu'aux flottants natifs est essentiel : il évite les erreurs d'arrondi qui seraient inacceptables sur des montants financiers.

2 Libération d'un jalon et répartition

À l'approbation d'un jalon, le service draine le séquestre, prélève la commission de la plateforme et crédite le freelance du montant net — chaque mouvement donnant lieu à une écriture distincte et idempotente.

app/Services/LedgerService.php — releaseMilestone() (extrait simplifié)

```

$amount      = (float) $milestone->amount;
$feeRate      = (float) PlatformSetting::get('fee.freelancer_pct', 0.10); // 10 %
$commission   = round($amount * $feeRate, 2);
$payout       = round($amount - $commission, 2);

// 1. Drainer le séquestre (client)           → transaction type=release
// 2. Créditer la commission (plateforme)      → transaction type=fee
// 3. Créditer le freelance (net)              → transaction type=credit
// 4. Marquer le jalon comme payé
$milestone->update(['status' => 'paid', 'approved_at' => now()]);

```

Des **garde-fous** protègent l'opération : un jalon doit être *soumis* avant d'être libéré, un contrat *en litige* bloque toute libération, et un séquestre insuffisant interrompt la transaction. Ces vérifications sont effectuées **sous verrou**, à l'abri des accès concurrents.

IV. Authentification et sécurité

Les routes sensibles sont protégées dès leur déclaration. La limitation de débit, par exemple, est appliquée par middleware directement sur les groupes de routes d'authentification.

routes/api.php — limitation de débit sur l'authentification (extrait)

```
Route::prefix('auth')->middleware('throttle:10,1')->group(function () {
    Route::post('/register', [AuthController::class, 'register']);
    Route::post('/login', [AuthController::class, 'login']);
});

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/auth/me', [AuthController::class, 'me']);
    // ... toutes les routes protégées exigent un jeton Bearer valide
});
```

La connexion vérifie les identifiants, applique éventuellement la 2FA, puis émet un jeton personnel que le client conservera pour authentifier ses requêtes suivantes.

app/Http/Controllers/API/Auth/AuthController.php — login() (extrait)

```
public function login(LoginRequest $request)
{
    $user = User::where('email', $request->email)->first();
    if (! $user || ! Hash::check($request->password, $user->password)) {
        return response()->json(['message' => 'Identifiants invalides'], 401);
    }
    $token = $user->createToken('spa')->plainTextToken; // jeton Sanctum
    return response()->json(['user' => $user, 'token' => $token]);
}
```

Les autorisations fines reposent sur des **politiques** Laravel. Ainsi, seules certaines actions — comme la résolution d'un litige — sont réservées à l'administrateur, et la libération d'un jalon est strictement réservée au client du contrat concerné. Cette vérification est dupliquée dans le service métier (défense en profondeur).

app/Policies/ContractPolicy.php — autorisation (extrait)

```
class ContractPolicy
{
    public function fund(User $user, Contract $contract): bool {
        return $user->id === $contract->client_id; // seul le client finance
    }
    public function resolveDispute(User $user, Contract $contract): bool {
        return $user->role === 'admin'; // arbitrage réservé à l'admin
    }
}
```

La validation des entrées est centralisée dans des **FormRequests**, qui définissent les règles de validation et les messages d'erreur, en amont du contrôleur.

app/Http/Requests/StoreProposalRequest.php — validation (extrait)

```
public function rules(): array {
    return [
        'cover_letter' => ['required', 'string', 'min:50', 'max:5000'],
        'bid_amount'   => ['required', 'numeric', 'min:5'],
    ];
}
```

V. Communication temps réel : la messagerie

La messagerie instantanée s'appuie sur **Laravel Reverb**, un serveur WebSocket. Lorsqu'un message est envoyé, un **événement** de diffusion est émis ; le serveur le pousse vers les participants de la conversation, que le client React reçoit via **Laravel Echo** — sans rechargement ni interrogation périodique.

Diffusion d'un message en temps réel (principe)

```
class MessageSent implements ShouldBroadcast
{
    public function __construct(public Message $message) {}

    public function broadcastOn(): array {
        return [new PrivateChannel('conversation.'.$this->message->conversation_id)];
    }
}
// Côté React : echo.private(`conversation.${id}`).listen('MessageSent', cb)
```

La messagerie gère en outre les **accusés de réception** et de lecture, les **réactions**, l'**édition** et la **suppression** de messages, ainsi que les **pièces jointes** — autant de fonctionnalités attendues d'un outil de communication moderne.

Côté React, un *hook* s'abonne au canal privé de la conversation à l'affichage du composant et met à jour la liste des messages en temps réel, sans rechargement.

frontend-react — abonnement temps réel (extrait)

```
useEffect(() => {
    const channel = echo.private(`conversation.${id}`)
        .listen('MessageSent', (e) => setMessages((m) => [...m, e.message]));
    return () => echo.leave(`conversation.${id}`); // nettoyage au démontage
}, [id]);
```

VI. Intégration des paiements Stripe

Stripe gère l'encaissement réel des dépôts. La confirmation arrive par un **webhook**, point d'entrée public mais sécurisé : sa signature est vérifiée, et son idempotence est garantie par la table **stripe_webhook_events**, qui empêche le double traitement d'un même

événement.

Traitement idempotent d'un webhook Stripe (principe)

```
// 1. Vérifier la signature de l'événement (secret du webhook)
$event = Webhook::constructEvent($payload, $sigHeader, $secret);

// 2. Idempotence : ignorer un événement déjà traité
if (StripeWebhookEvent::where('event_id', $event->id)->exists()) {
    return response('déjà traité', 200);
}
StripeWebhookEvent::create(['event_id' => $event->id, 'type' => $event->type]);

// 3. Créditer le portefeuille via le LedgerService (jamais en direct)
$Ledger->deposit($user, $amount, idempotencyKey: $event->id);
```

Les versements (retraits) suivent un parcours symétrique : le freelance configure son compte **Stripe Connect**, l'administrateur approuve la demande, puis un transfert est déclenché ; le webhook de confirmation fait basculer le retrait à l'état *complété*. En cas d'échec du transfert, les fonds sont automatiquement re-crédités au solde disponible.

VII. Intelligence artificielle

Panda intègre un assistant fondé sur un **modèle de langage** (Mistral, servi par Ollama). Cinq services sont exposés : génération de proposition, mise en correspondance freelances/offre, analyse de profil, recherche intelligente et assistant conversationnel. Chaque appel est limité en débit (protection des coûts) et historisé dans **ai_histories**.

app/Http/Controllers/API/AI/AIController.php — génération (principe)

```
public function generateProposal(Request $r)
{
    $prompt = $this->buildPrompt($r->job, $r->user->freelancerProfile);
    $text    = $this->ai->complete($prompt);           // appel au modèle (Ollama/Mistral)
    AiHistory::create([
        'user_id' => $r->user->id, 'type' => 'proposal',
        'input' => $prompt, 'output' => $text, 'tokens_used' => $tokens,
    ]);
    return response()->json(['proposal' => $text]);
}
```

Les cinq services d'IA exposés par la plateforme sont récapitulés ci-dessous.

Service	Description
Génération de proposition	Rédige une première version de candidature à partir de l'offre et du profil
Mise en correspondance	Classe les freelances les plus pertinents pour une offre donnée
Analyse de profil	Évalue un profil et suggère des améliorations
Recherche intelligente	Interprète une requête en langage naturel
Assistant conversationnel	Répond aux questions et guide l'utilisateur

Tableau 25: Services d'intelligence artificielle de la plateforme

VIII. Présentation des interfaces

Cette section parcourt les principales interfaces de Panda, conçues avec React et TailwindCSS dans un souci de clarté, de cohérence et de responsivité. *(Les captures présentées sont des maquettes fidèles de l'application.)*

1 Page d'accueil et pages publiques

La page d'accueil présente la proposition de valeur de la plateforme, oriente le visiteur vers les deux parcours (trouver un talent / trouver une mission) et met en avant les garanties de sécurité. Des pages publiques (tarifs, entreprises, ressources) complètent la vitrine.

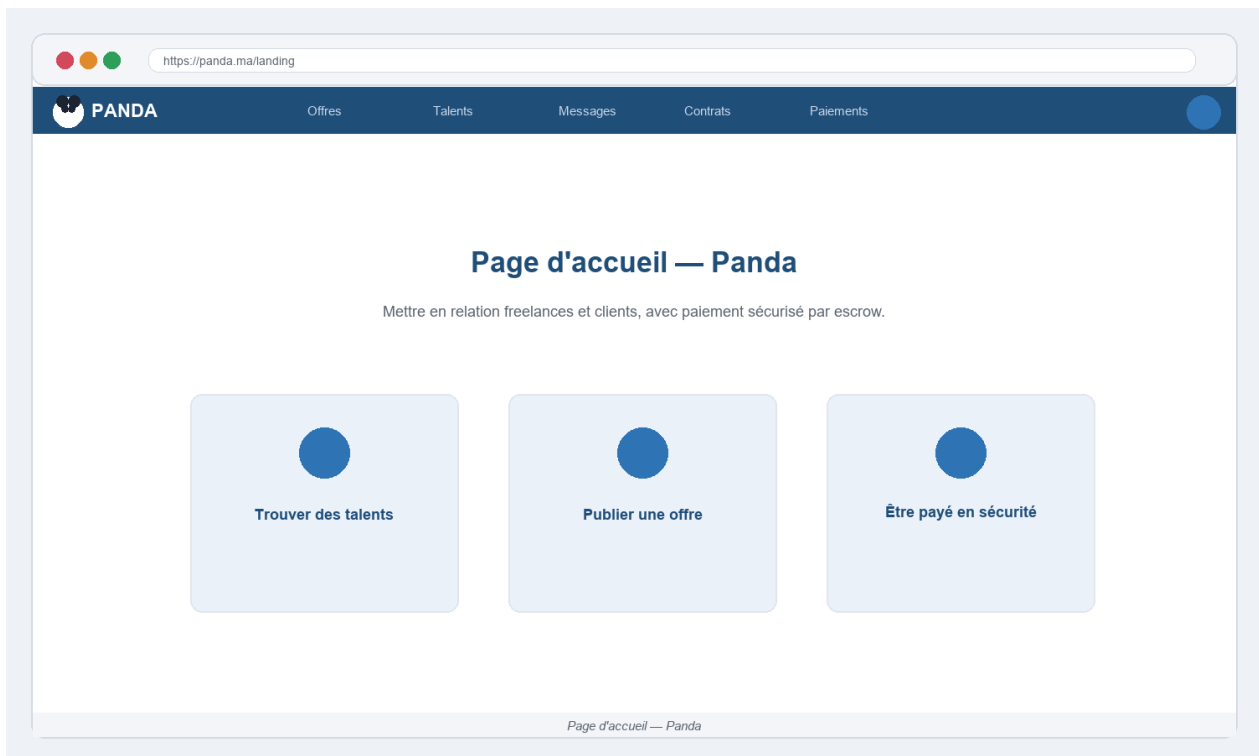


Figure 13: Page d'accueil de la plateforme Panda

Cette page met en avant :

- une **barre de recherche** orientant immédiatement vers les offres ou les talents ;
- des **indicateurs de confiance** (nombre de freelances, taux de satisfaction, paiement sécurisé) ;
- une présentation des **catégories** de compétences les plus demandées ;
- des appels à l'action clairs (« Trouver un talent », « Trouver une mission ») ;
- un **pied de page** riche : ressources, tarifs, entreprises, mentions légales.

2 Authentification et inscription

Les écrans d'authentification proposent la connexion classique, la connexion via Google et la création de compte avec choix du rôle (client ou freelance). La 2FA et la vérification d'identité renforcent ensuite la sécurité du compte.

The screenshot shows a web browser window with the URL `https://panda.ma/auth`. The page has a dark blue header with the PANDA logo and navigation links: Offres, Talents, Messages, Contrats, and Paiements. The main content area is white and contains a centered login form titled "Page de connexion" with the subtitle "Accédez à votre espace Panda". The form includes two input fields: "Adresse e-mail" and "Mot de passe". Below these is a blue button labeled "Se connecter". At the bottom of the form is a button labeled "Continuer avec Google". The footer of the page is light blue and contains the text "Page de connexion".

Figure 14: Écran de connexion (e-mail/mot de passe et OAuth Google)

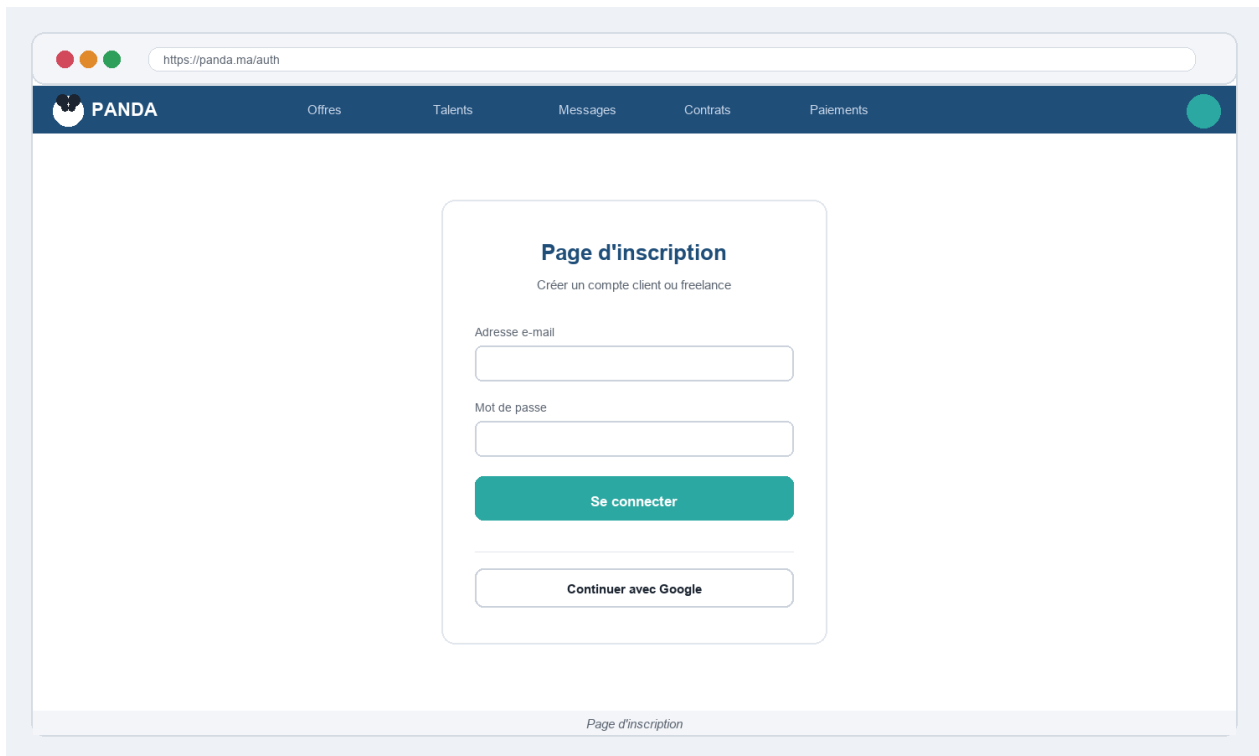


Figure 15: Écran d'inscription avec choix du rôle

Le parcours d'authentification intègre :

- la **connexion** par e-mail/mot de passe avec validation et messages d'erreur explicites ;
- la **connexion sociale** via Google (OAuth) en un clic ;
- l'**inscription** avec choix du rôle (client ou freelance) et confirmation du mot de passe ;
- la **réinitialisation** du mot de passe par e-mail ;
- l'activation de la **2FA** et la **vérification d'identité** depuis les paramètres du compte.

3 Tableaux de bord

Chaque rôle dispose d'un tableau de bord adapté, synthétisant les indicateurs clés et donnant accès aux actions courantes. Le client suit ses offres, contrats et dépenses ; le freelance suit ses missions, gains et propositions ; l'administrateur pilote la plateforme.

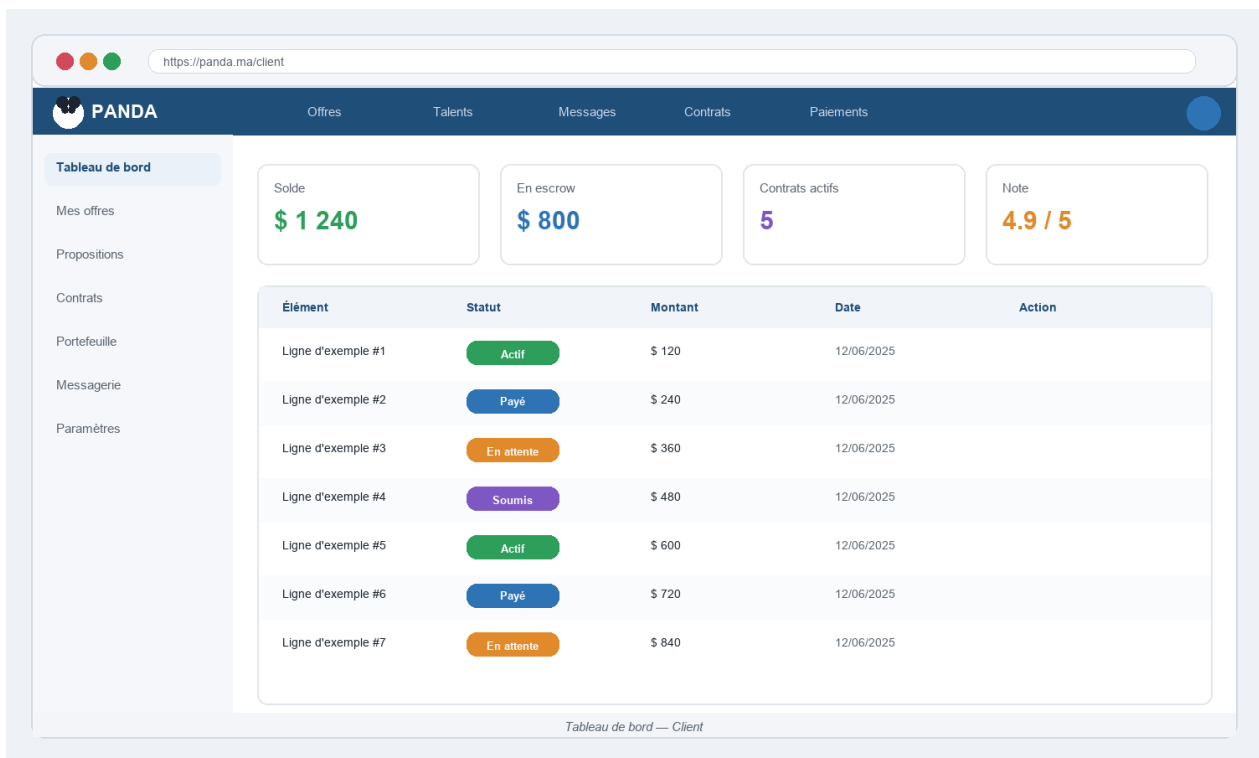


Figure 16: Tableau de bord du client

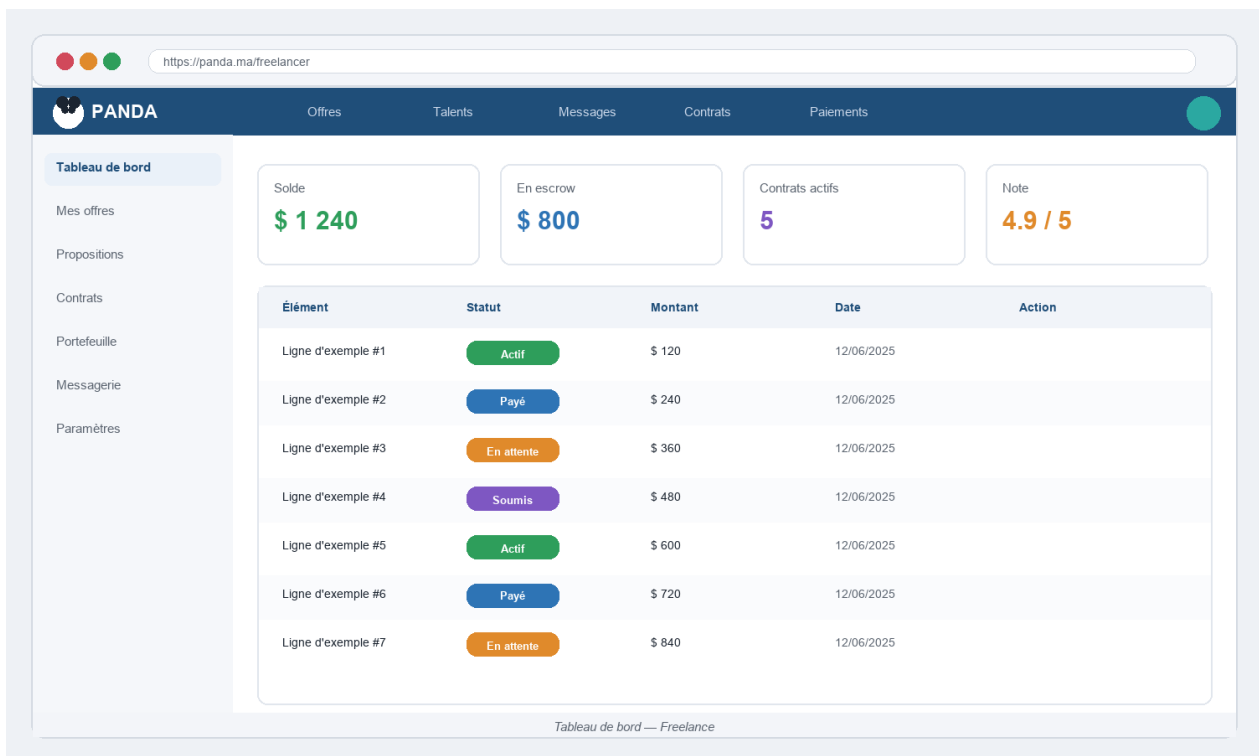


Figure 17: Tableau de bord du freelance

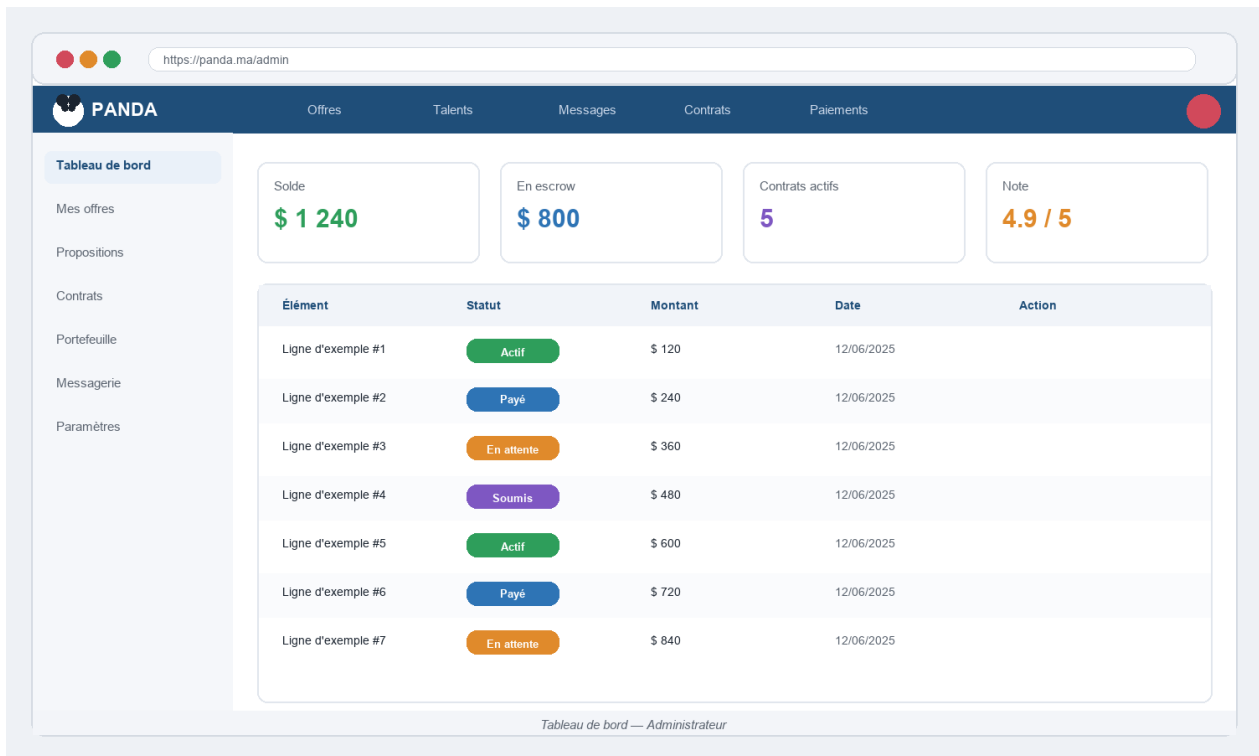


Figure 18: Tableau de bord de l'administrateur

Les indicateurs et actions diffèrent selon le rôle :

- **Client** : solde, fonds en séquestre, contrats actifs, offres publiées et propositions reçues ;
- **Freelance** : gains, propositions envoyées, contrats en cours, note moyenne et missions livrées ;
- **Administrateur** : utilisateurs, transactions, retraits en attente, files de modération et de vérification d'identité, indicateurs financiers de la plateforme.

4 Offres et propositions

La place de marché des offres permet de rechercher et de filtrer les missions par catégorie, budget et compétences. Le client publie une offre via un formulaire validé ; le freelance consulte le détail et soumet une proposition, éventuellement assistée par l'IA.

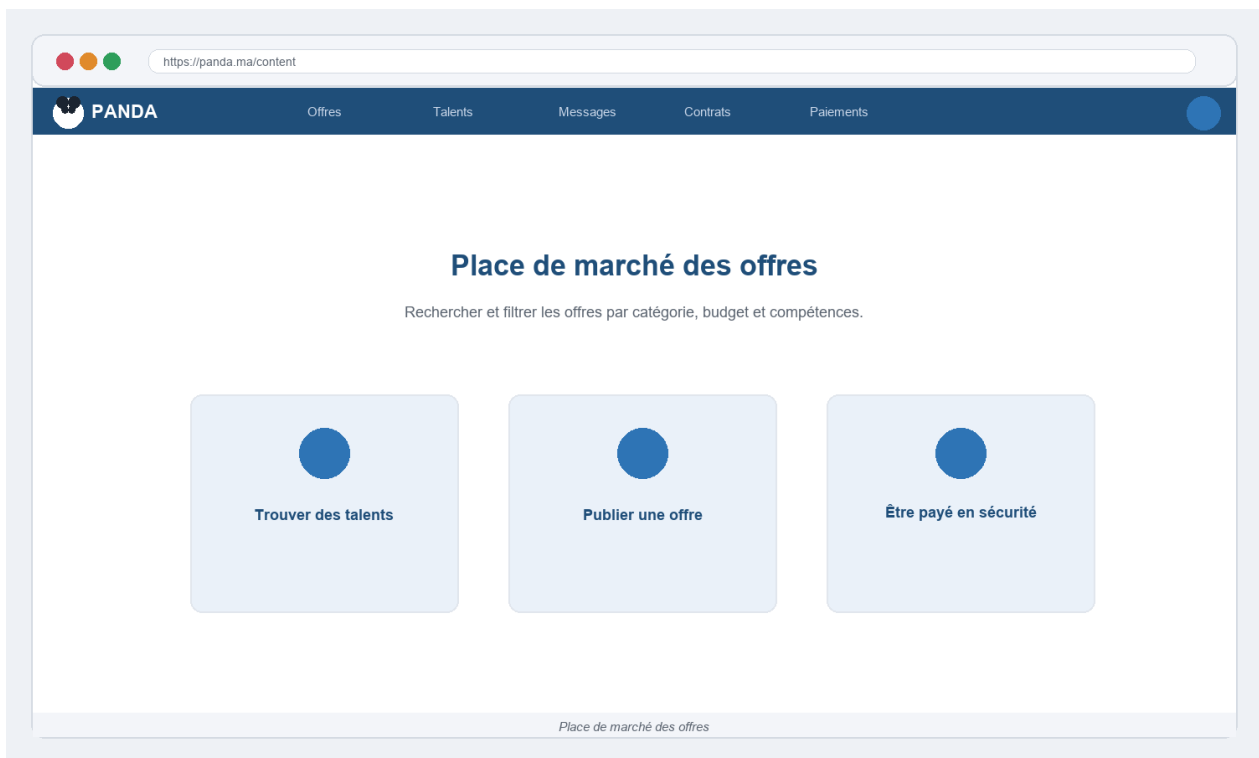


Figure 19: Place de marché des offres

- **recherche plein-texte** et filtres (catégorie, budget, type, compétences) ;
- affichage des offres sous forme de **cartes** synthétiques avec statut ;
- page de **détail** présentant la mission, le budget et les propositions ;
- **enregistrement** d'une offre en favori pour y revenir plus tard ;
- soumission d'une **proposition**, avec assistance optionnelle de l'IA.

5 Freelances et profils

La place de marché des talents met en valeur les profils des freelances : compétences, portfolio, évaluations et tarifs. Le client peut enregistrer ses favoris et constituer des listes de talents pour ses futurs projets.

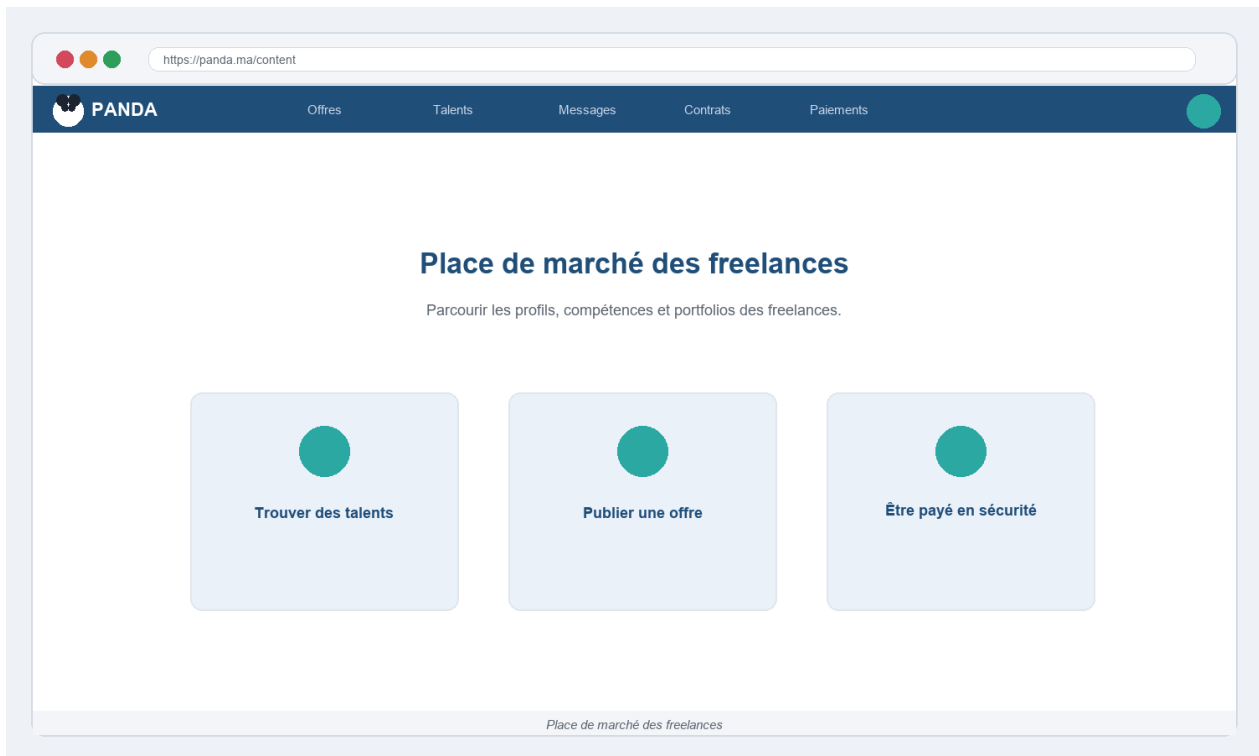


Figure 20: Place de marché des freelances

- fiches freelances avec **photo, titre, compétences, tarif** et note moyenne ;
- filtres par **compétence**, disponibilité et fourchette de tarif ;
- page de **profil détaillé** : biographie, portfolio, évaluations reçues ;
- ajout aux **favoris** et constitution de **listes de talents** nommées ;
- lancement direct d'une **conversation** ou d'une proposition de contrat.

6 Contrats et jalons

La page de détail d'un contrat centralise l'ensemble de la collaboration : jalons, fichiers versionnés, suivi du temps, demandes d'extension, fil d'activité, analyse et messagerie contextuelle. Le client finance le séquestre et valide les jalons ; le freelance livre son travail.

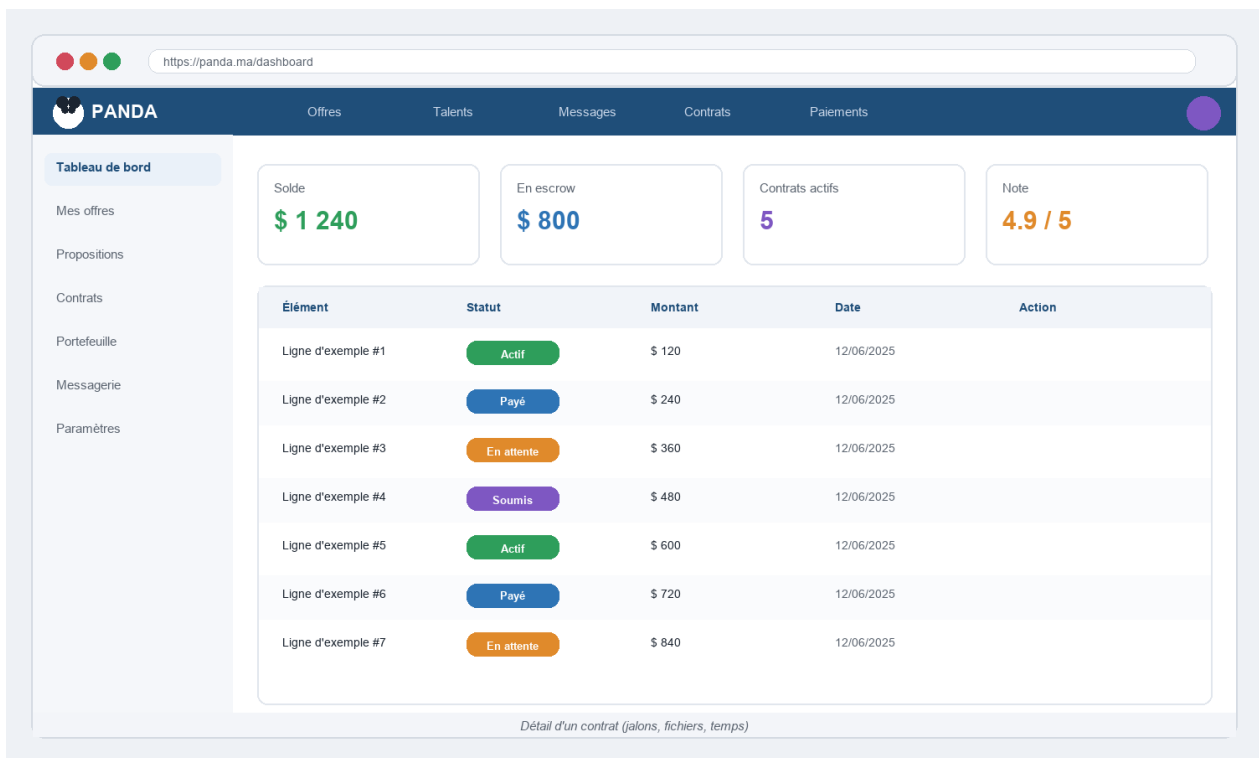


Figure 21: Détail d'un contrat — jalons, fichiers et suivi du temps

La page de contrat s'organise en onglets :

- **Jalons** : création, soumission, validation/rejet, suivi de l'avancement ;
- **Fichiers** : dépôt, versionnement et téléchargement des livrables ;
- **Temps** : chronomètre de suivi et récapitulatif hebdomadaire (contrats horaires) ;
- **Extensions** : demande et réponse de prolongation de délai ;
- **Activité** : journal chronologique des événements du contrat ;
- **Analyse et messagerie** contextuelle, plus génération du **PDF** du contrat.

7 Paiements et portefeuille

L'espace de paiement présente le portefeuille (solde disponible, en séquestre, en attente), l'historique des transactions, le dépôt de fonds via Stripe et la demande de retrait. Chaque mouvement est tracé et réconciliable.

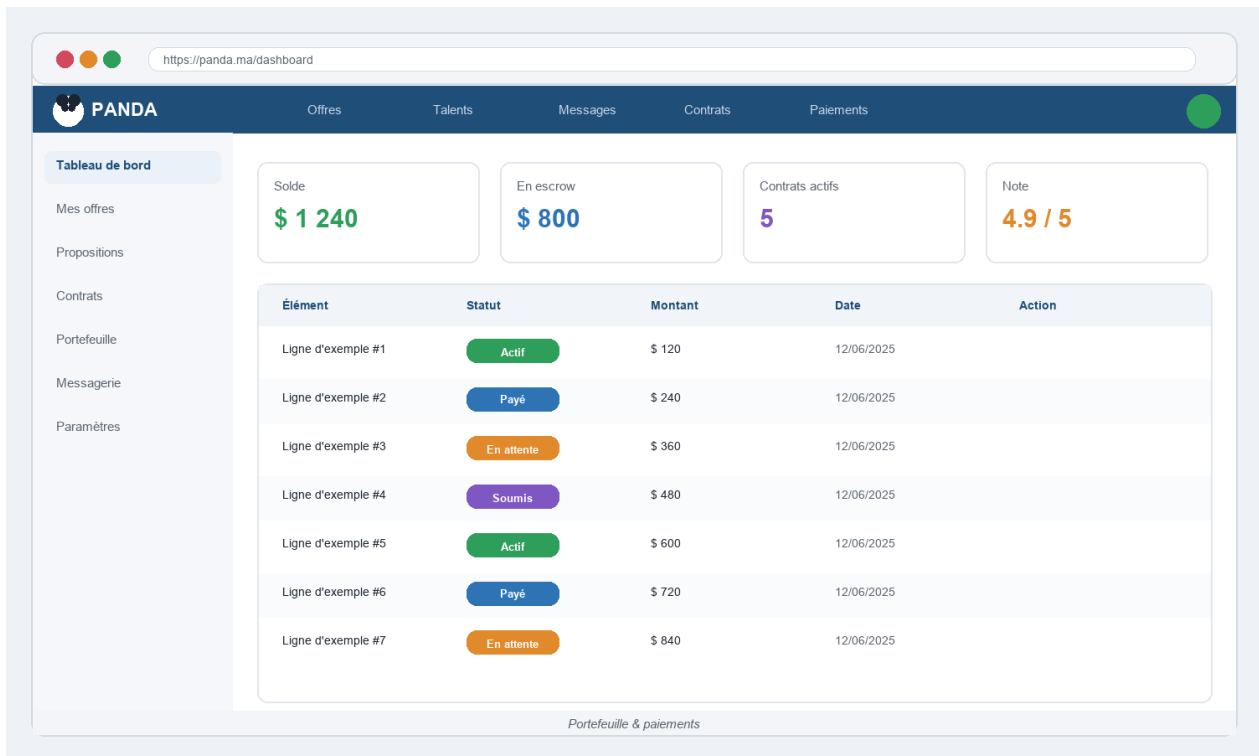


Figure 22: Portefeuille et gestion des paiements

- trois soldes distincts : **disponible**, **en séquestre** et **en attente** ;
- **dépôt** de fonds via Stripe Checkout et **retrait** vers Stripe Connect ;
- **historique** détaillé et filtrable de toutes les transactions ;
- suivi de l'état des **retraits** (en attente, approuvé, complété, rejeté) ;
- factures hebdomadaires pour les **contrats horaires** et les abonnements.

8 Messagerie

La messagerie temps réel relie clients et freelances autour des offres et des contrats. Elle affiche l'état de présence, les accusés de lecture et les réactions, et autorise le partage de pièces jointes.

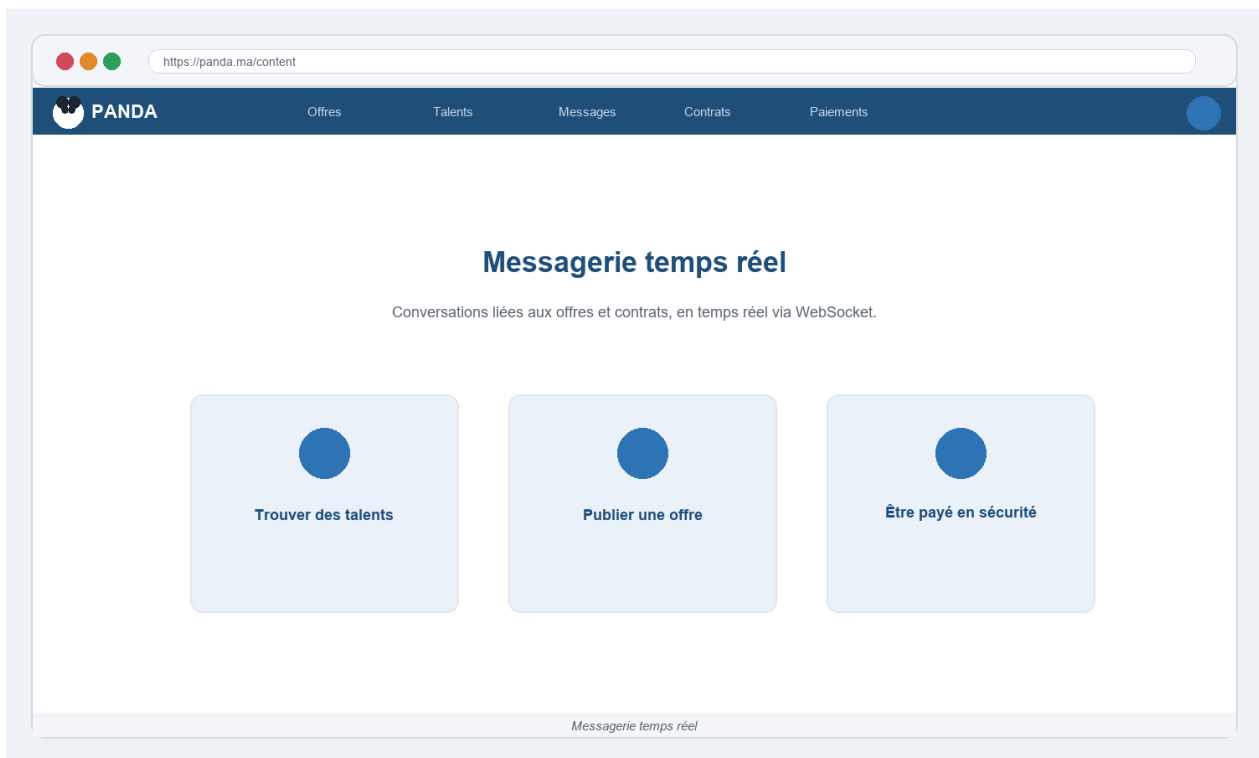


Figure 23: Messagerie temps réel

- liste des **conversations** triées par activité récente ;
- **indicateur de saisie** (« en train d'écrire ») et statut de présence ;
- **accusés** de réception et de lecture des messages ;
- **réactions**, édition et suppression des messages ;
- partage de **pièces jointes** et conversations liées aux offres et contrats.

9 Assistant IA

L'assistant IA accompagne l'utilisateur : rédaction de propositions, mise en correspondance entre une offre et les profils les plus pertinents, et analyse de profil. Il accélère les tâches répétitives tout en laissant l'utilisateur maître du résultat final.

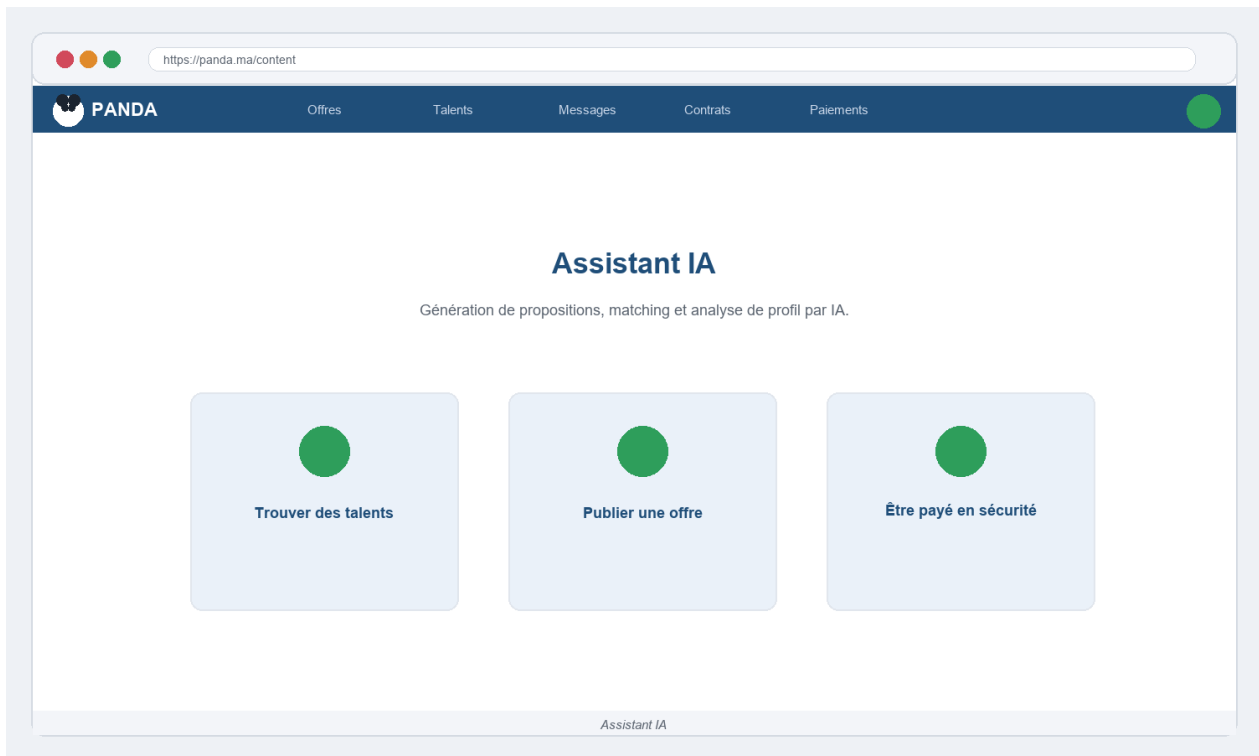


Figure 24: Assistant d'intelligence artificielle

- **génération de proposition** à partir d'une offre et du profil du freelance ;
- **mise en correspondance** d'une offre avec les freelances les plus pertinents ;
- **analyse de profil** et suggestions d'amélioration ;
- **recherche intelligente** en langage naturel ;
- **assistant conversationnel** pour guider l'utilisateur □historisation des échanges.

10 Design responsive et expérience utilisateur

Une attention constante a été portée à l'**expérience utilisateur**. Construites avec TailwindCSS, toutes les interfaces sont **responsives** et s'adaptent harmonieusement au téléphone, à la tablette et à l'ordinateur. Au-delà de l'adaptabilité, plusieurs principes ont guidé la conception de l'interface :

- **Cohérence visuelle** : une charte graphique unifiée (couleurs, typographie, composants) ;
- **Retours immédiats** : notifications *toast*, états de chargement et messages d'erreur clairs ;
- **Navigation intuitive** : barre de navigation, menu latéral contextuel et fil d'Ariane ;
- **Animations mesurées** (Framer Motion) renforçant la lisibilité sans distraire ;
- **Visualisation de données** (Recharts) pour les tableaux de bord et les statistiques ;

- **Accessibilité** : contrastes suffisants, navigation au clavier et libellés explicites.

Ce soin apporté à l'interface est déterminant : sur une place de marché, la qualité de l'expérience conditionne directement la confiance et l'engagement des utilisateurs.

IX. Conclusion du chapitre

Ce chapitre a illustré la réalisation de Panda, de l'organisation du code aux interfaces, en passant par les briques techniques les plus exigeantes. Le moteur financier, la sécurité, le temps réel, l'intégration de Stripe et l'IA ont été présentés à travers des extraits de code représentatifs. La plateforme obtenue est fonctionnelle, cohérente et fidèle à la conception. Le chapitre suivant aborde la **validation** de ce travail : tests, sécurité et déploiement.

Chapitre 5 : Tests, sécurité et déploiement

La qualité d'une plateforme transactionnelle ne se mesure pas seulement à ses fonctionnalités, mais à sa **fiabilité**, à sa **sécurité** et à sa capacité à être **déployée** sereinement. Ce chapitre présente la stratégie de tests adoptée, en insistant sur la validation du moteur financier, fait la synthèse des mesures de sécurité, puis décrit l'architecture de déploiement conteneurisée.

I. Stratégie de tests

Nous avons adopté une stratégie de tests **automatisés** centrée sur les comportements critiques, à l'aide de **PHPUnit** (intégré à Laravel). Deux grandes familles de tests ont été écrites : des **tests unitaires**, qui vérifient une unité de logique isolée (un service, une méthode), et des **tests fonctionnels** (*feature tests*), qui exercent un point d'API de bout en bout — de la requête HTTP à la base de données.

- **Tests d'authentification** : inscription, connexion, limitation de débit, 2FA, réinitialisation de mot de passe ;
- **Tests du flux métier** : publication d'offre, soumission et acceptation de proposition, création de contrat, cycle de vie des jalons ;
- **Tests du moteur financier** : dépôt, séquestre, libération, retrait, remboursement, litige ;
- **Tests d'autorisation** : un utilisateur ne peut agir que dans le périmètre de ses droits ;
- **Tests de validation** : rejet des entrées invalides par les FormRequests.

Le tableau ci-dessous récapitule la couverture par domaine fonctionnel et le type de tests associé.

Domaine	Type de tests	Priorité
Authentification & sécurité	Fonctionnels + unitaires	Élevée
Moteur financier (ledger, escrow)	Unitaires + fonctionnels	Critique
Contrats & jalons	Fonctionnels	Élevée
Offres, propositions, catalogue	Fonctionnels	Moyenne
Messagerie & notifications	Fonctionnels	Moyenne
Autorisation (policies)	Fonctionnels	Élevée
Validation des entrées	Unitaires	Moyenne

Tableau 26: Couverture des tests par domaine fonctionnel

II. Validation du moteur financier

Le **LedgerService** a fait l'objet d'une attention toute particulière, car la moindre faille y serait critique. Au-delà des cas nominaux, les tests vérifient des **invariants** et des **scénarios limites** : que se passe-t-il en cas d'appel dupliqué, de solde insuffisant, de double approbation, ou de contrat en litige ?

Exemple de test fonctionnel — la libération crédite le bon montant net (principe)

```
public function test_release_credite_le_freelance_du_montant_net(): void
{
    // étant donné un séquestre financé et un jalon soumis de 100
    $this->ledger->fundEscrow($client, $contract, 100);
    $milestone = Milestone::factory()->submitted()->create(['amount' => 100]);

    // quand le client libère le jalon
    $this->ledger->releaseMilestone($client, $milestone);

    // alors : freelance +90, plateforme +10, jalon = payé, séquestre = 0
    $this->assertEquals(90, $freelancer->wallet->balance);
    $this->assertEquals('paid', $milestone->fresh()->status);
}
```

Le tableau suivant récapitule un échantillon des cas de test du moteur financier et leurs résultats.

Cas de test	Attendu	Résultat
Dépôt de fonds	Solde crédité, transaction tracée	Conforme
Financement du séquestre	balance ↓, escrow ↑, atomique	Conforme
Libération de jalon (commission 10 %)	Freelance +net, plateforme +frais	Conforme
Appel dupliqué (même clé)	Aucune double écriture (idempotent)	Conforme
Solde insuffisant	Opération refusée, état inchangé	Conforme
Libération sur contrat en litige	Bloquée par garde-fou	Conforme
Retrait : demande puis rejet admin	Fonds re-crédités au solde	Conforme
Invariant $\Sigma(\text{transactions}) = \text{solde}$	Égalité vérifiée	Conforme

Tableau 27: Échantillon des cas de test du moteur financier

Résultat : l'ensemble des comportements financiers critiques se comporte conformément aux attentes, y compris dans les scénarios concurrents et d'erreur. La cohérence des soldes est garantie par construction (transactions, verrous, double-entrée).

III. Synthèse de la sécurité

La sécurité de Panda repose sur une **défense en profondeur** : plutôt qu'une barrière unique, plusieurs couches indépendantes se renforcent mutuellement. Le tableau suivant synthétise les mesures mises en œuvre et la menace qu'elles adressent.

Mesure	Menace adressée
Hachage bcrypt des mots de passe	Fuite de la base / vol d'identifiants
Jetons Sanctum + 2FA (TOTP)	Usurpation de compte
Limitation de débit (throttling)	Force brute, credential stuffing, abus d'API
Politiques d'autorisation (Policies)	Élévation de privilèges, accès non autorisé
Validation systématique (FormRequest)	Données malveillantes, injection
Requêtes préparées (Eloquent/PDO)	Injection SQL
Vérification de signature des webhooks	Falsification d'événements de paiement
Idempotence des opérations financières	Double paiement, rejeu
Vérification d'identité (KYC)	Comptes frauduleux, blanchiment
Journal d'audit	Traçabilité, investigation post-incident

Tableau 28: Synthèse des mesures de sécurité (défense en profondeur)

IV. Déploiement

Pour garantir la portabilité et la reproductibilité de l'environnement, Panda est conçu pour être **conteneurisé** avec **Docker**. Chaque service tourne dans son propre conteneur, orchestré par **Docker Compose**, ce qui élimine les divergences entre les postes de développement et la production.

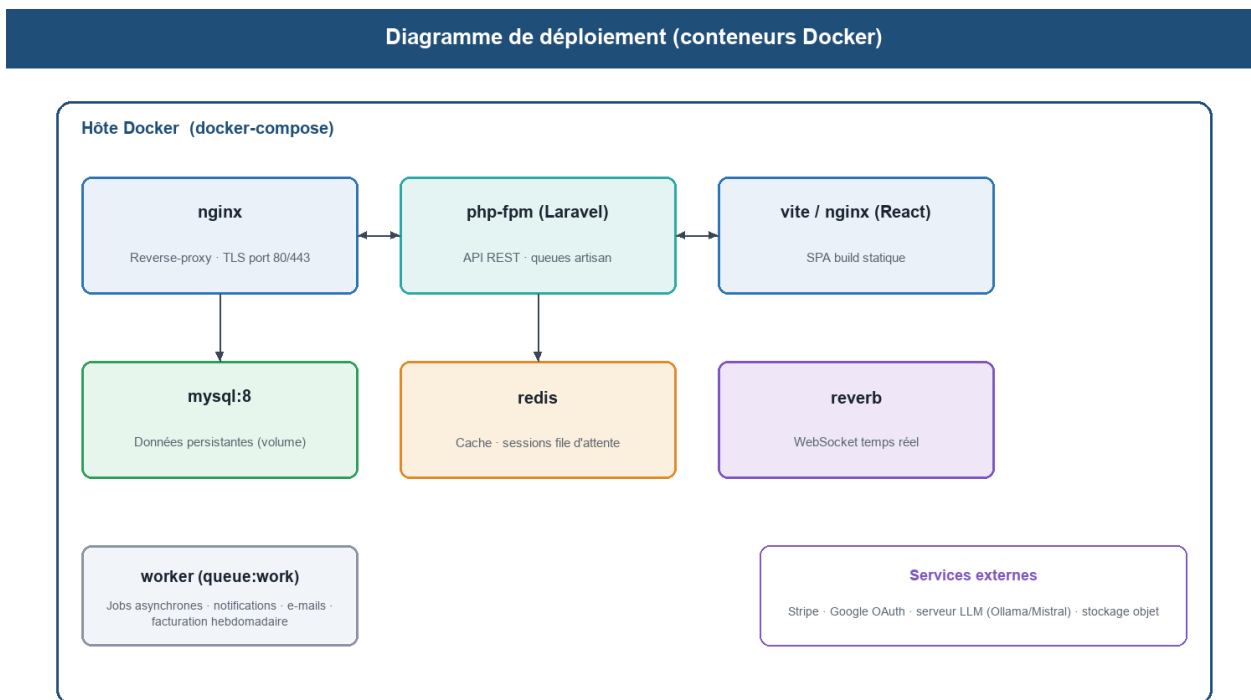


Figure 25: Diagramme de déploiement — conteneurs Docker

L'architecture de déploiement comprend : un **reverse-proxy** (nginx) assurant le routage et le TLS, le conteneur **applicatif** (PHP-FPM/Laravel) servant l'API, le **front-end** React compilé et servi statiquement, la base **MySQL** (sur volume persistant), **Redis** (cache et file d'attente), le serveur **Reverb** (WebSockets) et un **worker** traitant les tâches asynchrones (notifications, e-mails, facturation hebdomadaire).

1 Optimisations de performance

Plusieurs dispositifs concourent à de bonnes performances : **index plein-texte** MySQL pour accélérer la recherche d'offres et de freelances, **cache Redis** pour les données fréquemment consultées, **pagination** systématique des listes, **chargement anticipé** des relations Eloquent (pour éviter le problème des requêtes N+1), et exécution **asynchrone** des tâches lourdes (envoi d'e-mails, notifications, facturation) via une file d'attente. Le découplage front/back et

la conception sans état de l'API facilitent par ailleurs une éventuelle montée en charge horizontale.

2 Intégration et déploiement continus

Le dépôt est outillé pour l'**intégration continue** : à chaque évolution, la suite de tests est exécutée automatiquement, garantissant qu'aucune régression n'est introduite. Cette discipline, associée au versionnement Git, sécurise l'évolution du code et prépare un **déploiement continu** vers un environnement d'hébergement.

Mise en service (principe)

```
docker compose up -d --build      # démarrage de la pile
php artisan migrate --force        # application des migrations
php artisan queue:work             # traitement des tâches asynchrones
php artisan reverb:start           # serveur WebSocket temps réel
npm run build                      # build du front-end React
```

Avant chaque livraison, une phase de **recette** vérifie manuellement les parcours critiques de bout en bout (inscription → publication d'offre → proposition → contrat → séquestre → libération), en complément des tests automatisés. La combinaison des tests de **non-régression** (rejoués à chaque évolution) et de la recette fonctionnelle garantit qu'une nouvelle fonctionnalité n'altère pas les comportements existants, condition essentielle de la confiance sur une plateforme financière.

V. Conclusion du chapitre

Ce chapitre a démontré que Panda n'est pas seulement fonctionnelle, mais **fiable**, **sécurisée** et **déployable**. La stratégie de tests, focalisée sur le moteur financier, a confirmé l'intégrité des opérations critiques. La synthèse de la sécurité a mis en évidence une défense en profondeur cohérente. Enfin, la conteneurisation Docker garantit un déploiement reproductible. Le dernier chapitre dresse le bilan global du projet et ouvre sur ses perspectives.

Chapitre 6 : Bilan et perspectives

Ce dernier chapitre prend du recul sur le projet. Il dresse le bilan des compétences acquises, revient sur les difficultés rencontrées et les contraintes du projet, puis ouvre sur les perspectives d'évolution de la plateforme.

I. Bilan des compétences techniques acquises

La réalisation de Panda, par son ampleur, a constitué un terrain d'apprentissage exceptionnel. Elle nous a permis de consolider et d'approfondir un large éventail de compétences techniques :

- **Conception logicielle** : modélisation UML (cas d'utilisation, classes, séquence, états) et conception de bases de données relationnelles avec Merise ;
- **Développement back-end** : maîtrise du framework Laravel, conception d'une API REST, patron MVC enrichi de services métier, ORM Eloquent, migrations versionnées ;
- **Développement front-end** : application monopage React, gestion d'état (Zustand), routage, consommation d'API, design responsive avec TailwindCSS ;
- **Sécurité applicative** : authentification par jeton, 2FA, OAuth, politiques d'autorisation, limitation de débit, KYC, journal d'audit ;
- **Intégrité transactionnelle** : conception d'un grand livre à écriture double, transactions, verrouillage, idempotence et précision décimale ;
- **Intégration de services tiers** : Stripe (paiements et versements), Google OAuth, modèle de langage pour l'IA, Web Push ;
- **Temps réel** : diffusion d'événements via WebSockets (Laravel Reverb / Echo) ;
- **Industrialisation** : versionnement Git, tests automatisés, conteneurisation Docker.

Plus encore que la maîtrise d'outils isolés, ce projet nous a appris à **assembler** un écosystème technique cohérent : faire dialoguer un front-end et un back-end découplés, intégrer des services tiers hétérogènes, et garantir la cohérence d'un ensemble complexe. Cette vision d'architecte, qui dépasse la simple écriture de code, constitue sans doute l'acquis le plus précieux de cette expérience.

II. Compétences transversales

Au-delà de la technique, ce projet a renforcé des compétences humaines et méthodologiques essentielles à la vie professionnelle :

- Analyse et formalisation d'un besoin complexe en un cahier des charges structuré ;
- Gestion de projet agile : découpage en incréments, priorisation par la valeur et le risque ;
- Gestion du temps et respect d'échéances, en parallèle de la formation ;
- Capacité d'adaptation et résolution autonome de problèmes techniques pointus ;
- Rédaction d'une documentation technique claire et d'un rapport structuré ;
- Esprit de synthèse et prise de décision face aux arbitrages de périmètre.

III. Difficultés rencontrées

Comme tout projet d'envergure, Panda a présenté plusieurs défis qui ont nécessité recherche, réflexion et rigueur :

- **Intégrité financière** : garantir qu'aucune opération concurrente ne puisse corrompre les soldes a imposé une étude approfondie des transactions et des verrous SQL ;
- **Idempotence** : concevoir des opérations rejouables sans effet de bord, indispensables face aux webhooks et aux re-tentatives réseau ;
- **Communication temps réel** : la mise en place du serveur WebSocket et la synchronisation de l'état côté React ont demandé une bonne compréhension du modèle événementiel ;
- **Ampleur du périmètre** : la richesse fonctionnelle a exigé une organisation rigoureuse pour éviter la dette technique et préserver la cohérence de l'ensemble ;
- **Intégration de l'IA** : la maîtrise des coûts et la gestion des temps de réponse du modèle ont nécessité limitation de débit et historisation.

Chaque difficulté a été l'occasion d'un apprentissage durable. La rigueur imposée par le moteur financier, en particulier, nous a sensibilisés à un niveau d'exigence propre aux applications critiques.

IV. Contraintes et limites

Le projet a été mené dans un cadre académique, avec ses contraintes propres : un **temps limité**, mené en parallèle de la formation, et l'absence d'un environnement de production réel

à grande échelle. Certaines fonctionnalités, bien qu'esquissées ou conçues, n'ont pas été menées à un niveau de finition complet : la gestion fine des litiges multi-jalons, l'internationalisation multi-devises, ou encore certaines optimisations de performance sous forte charge. Ces limites n'enlèvent rien à la cohérence de l'ensemble et constituent autant de pistes d'amélioration.

V. Perspectives d'évolution

Panda a été conçue pour évoluer. Plusieurs axes d'enrichissement se dessinent naturellement :

- **Application mobile native** (iOS/Android) consommant l'API existante ;
- **Internationalisation** : multi-langues et multi-devises, avec conversion et conformité fiscale ;
- **Recommandation avancée** : moteur de mise en correspondance plus fin, fondé sur l'IA et l'historique des collaborations ;
- **Tableau de bord analytique** enrichi pour les freelances, les agences et l'administration ;
- **Programme de confiance** : badges, niveaux, garanties et résolution de litiges assistée ;
- **Passage à l'échelle** : mise en cache avancée, montée en charge horizontale, observabilité (métriques, traces, alertes).

VI. Conclusion du chapitre

Ce chapitre a dressé un bilan globalement très positif : Panda a permis de mobiliser un large spectre de compétences et de livrer une plateforme cohérente et ambitieuse. Les difficultés rencontrées ont nourri des apprentissages durables, et les limites identifiées ouvrent des perspectives d'évolution stimulantes. La conclusion générale qui suit synthétise l'apport de ce projet.

Conclusion générale

Au terme de ce projet de fin d'études, nous avons conçu et développé **Panda**, une plateforme web full-stack de mise en relation entre freelances et clients. Partant d'un constat simple — la difficulté de bâtir la confiance entre des parties distantes — nous avons construit une solution complète qui couvre l'ensemble du cycle de collaboration et place la **sécurité financière** au cœur de son architecture.

Sur le plan technique, le projet a permis de mettre en œuvre une architecture professionnelle moderne : une API REST Laravel découplée, une interface React réactive, une base de données MySQL rigoureusement modélisée, et un ensemble de services tiers intégrés (Stripe, OAuth, WebSockets, intelligence artificielle). La pièce maîtresse en est le **moteur de grand livre** à écriture double, transactionnel et idempotent, qui garantit l'intégrité de chaque mouvement de fonds — du dépôt au versement, en passant par le séquestre et la libération des jalons.

Au-delà du résultat technique, cette expérience nous a profondément formés. Elle nous a confrontés aux réalités d'un projet d'envergure : la nécessité d'une conception soignée, l'exigence de qualité propre aux applications critiques, et l'importance des arbitrages. Elle a consolidé nos compétences en développement full-stack et renforcé notre autonomie, notre capacité d'analyse et notre rigueur.

Ce projet illustre enfin une conviction qui a guidé l'ensemble de notre travail : la technologie, lorsqu'elle est mise au service d'un besoin réel, peut rendre le travail plus **juste** et plus **accessible**. En sécurisant la rémunération du freelance et en protégeant l'investissement du client, Panda ne se contente pas de mettre en relation deux parties : elle crée les conditions de la confiance, sans laquelle aucune collaboration durable n'est possible. C'est cette dimension — à la fois technique et humaine — qui donne tout son sens à ce projet.

Panda n'est pas une fin en soi, mais une fondation solide et évolutive. Les perspectives esquissées — application mobile, internationalisation, recommandation avancée, passage à l'échelle — témoignent du potentiel de la plateforme. Nous achevons ce travail avec la conviction d'avoir mené un projet à la fois ambitieux et abouti, et avec une vision plus claire et plus assurée du métier de développeur. Pour finir, nous adressons nos remerciements à

toutes les personnes qui nous ont accompagnés et soutenus tout au long de cette aventure.

Glossaire

Le tableau suivant définit les principaux termes techniques employés dans ce rapport.

Terme	Définition
API REST	Interface de programmation exposant des ressources via HTTP selon les principes REST.
SPA	Single Page Application : application web chargée une fois et mise à jour dynamiquement.
Escrow (séquestre)	Mécanisme par lequel des fonds sont bloqués par un tiers de confiance jusqu'à la réalisation d'une condition (livraison conforme).
Jalon (milestone)	Étape d'un contrat associée à un montant, soumise puis validée.
Grand livre (ledger)	Registre comptable où chaque mouvement est enregistré de façon immuable.
Écriture double	Principe comptable où tout débit a un crédit correspondant.
Idempotence	Propriété d'une opération produisant le même effet, qu'elle soit exécutée une ou plusieurs fois.
Jeton Bearer	Jeton d'accès transmis dans l'en-tête HTTP Authorization pour authentifier.
Sanctum	Système d'authentification par jeton de Laravel, adapté aux SPA.
2FA / TOTP	Authentification à deux facteurs par code temporel à usage unique.
OAuth	Protocole de délégation d'autorisation (connexion via Google, etc.).
KYC	Know Your Customer : vérification de l'identité d'un utilisateur.
Webhook	Requête HTTP émise par un service tiers (Stripe) pour notifier un événement.
WebSocket	Protocole de communication bidirectionnelle temps réel.
ORM	Object-Relational Mapping : correspondance entre objets et tables relationnelles.
Throttling	Limitation du nombre de requêtes par intervalle de temps.
Migration	Script versionné décrivant l'évolution du schéma de la base de données.

Tableau 29: Glossaire des principaux termes

Références bibliographiques et webographie

Les ressources suivantes ont été consultées tout au long de la conception et de la réalisation du projet.

- Laravel, *Documentation officielle*, <https://laravel.com/docs>
- Laravel Sanctum, *API Token Authentication*, <https://laravel.com/docs/sanctum>
- Laravel Reverb, *WebSockets temps réel*, <https://laravel.com/docs/reverb>
- React, *Documentation officielle*, <https://react.dev>
- Vite, *Next Generation Frontend Tooling*, <https://vitejs.dev>
- TailwindCSS, *Documentation*, <https://tailwindcss.com/docs>
- Zustand, *State management for React*, <https://zustand-demo.pmnd.rs>
- Stripe, *API Reference & Connect*, <https://stripe.com/docs>
- MySQL, *Reference Manual*, <https://dev.mysql.com/doc>
- Ollama, *Run large language models locally*, <https://ollama.com>
- OMG, *Unified Modeling Language (UML) Specification*, <https://www.omg.org/spec/UML>
- Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley.
- OpenClassrooms, *Concevez votre site web avec PHP et MySQL*, <https://openclassrooms.com>

Annexes

Cette section regroupe quelques éléments techniques complémentaires destinés à éclairer la mise en œuvre du projet.

A Configuration de l'environnement

La configuration de l'application est externalisée dans un fichier d'environnement, ce qui sépare le code des secrets et facilite le passage d'un environnement à l'autre. En voici un extrait représentatif (valeurs sensibles masquées).

backend-laravel/.env (extrait)

```
APP_NAME=Panda
APP_ENV=local
DB_CONNECTION=mysql
DB_DATABASE=panda
DB_USERNAME=root

SANCTUM_STATEFUL_DOMAINS=localhost:5173
BROADCAST_CONNECTION=reverb
REVERB_APP_KEY=*****

STRIPE_KEY=pk_test_*****
STRIPE_SECRET=sk_test_*****
STRIPE_WEBHOOK_SECRET=whsec_*****

GOOGLE_CLIENT_ID=*****
OLLAMA_BASE_URL=http://localhost:11434
```

B Exemple de réponse de l'API

Les réponses de l'API sont normalisées au format JSON. L'exemple suivant illustre le détail d'un portefeuille retourné par le point d'accès `/api/payments/wallet`.

Exemple de réponse JSON — portefeuille

```
{
  "wallet": {
    "balance": "1240.00",
    "escrow_balance": "800.00",
    "pending_balance": "0.00",
    "currency": "USD"
  },
  "recent_transactions": [
    { "reference": "PAY-3KX...", "type": "credit", "amount": "90.00" },
    { "reference": "ESC-9Q2...", "type": "escrow", "amount": "800.00" }
  ]
}
```

C Principaux services métier

Le tableau ci-dessous recense les services métier de l'application et leur responsabilité.

Service	Responsabilité
LedgerService	Tous les mouvements de fonds (grand livre à écriture double)
StripeService	Sessions de paiement, Connect et versements
SubscriptionService	Gestion des abonnements et des crédits
HourlyBillingService	Facturation hebdomadaire des contrats horaires
CatalogCheckoutService	Commandes du catalogue de services
NotificationService	Émission des notifications in-app et push
ContractActivityService	Journalisation des événements d'un contrat
AuditLogService	Traçabilité des actions sensibles
TotpService	Génération et vérification des codes 2FA

Tableau 30: Principaux services métier de la plateforme

D Commandes utiles

Quelques commandes Artisan et npm fréquemment utilisées durant le développement :

Commandes Artisan / npm

```
php artisan migrate           # appliquer les migrations
php artisan db:seed           # injecter les données de démonstration
php artisan test               # exécuter la suite de tests
php artisan queue:work         # traiter la file d'attente
php artisan reverb:start       # démarrer le serveur WebSocket
npm run dev                    # serveur de développement front-end
npm run build                  # build de production du front-end
```

E Extrait du script de création des tables financières

À titre d'illustration, voici le script SQL (généré à partir des migrations) des deux tables au cœur de l'intégrité financière, avec leurs types décimaux et leurs contraintes.

Schéma SQL — wallets & transactions (extrait)


```

CREATE TABLE wallets (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  user_id     BIGINT UNSIGNED NOT NULL,
  balance     DECIMAL(12,2) NOT NULL DEFAULT 0,
  pending_balance DECIMAL(12,2) NOT NULL DEFAULT 0,
  escrow_balance DECIMAL(12,2) NOT NULL DEFAULT 0,
  currency    VARCHAR(255) NOT NULL DEFAULT 'USD',
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE transactions (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  wallet_id   BIGINT UNSIGNED NOT NULL,
  user_id     BIGINT UNSIGNED NOT NULL,
  contract_id BIGINT UNSIGNED NULL,
  milestone_id BIGINT UNSIGNED NULL,
  reference    VARCHAR(255) NOT NULL UNIQUE,
  type        ENUM('credit','debit','escrow','release','refund','withdrawal','fee'),
  amount      DECIMAL(12,2) NOT NULL,
  fee         DECIMAL(12,2) NOT NULL DEFAULT 0,
  balance_after DECIMAL(12,2) NULL,
  idempotency_key VARCHAR(255) NULL,
  status      ENUM('pending','completed','failed','cancelled') DEFAULT 'pending',
  FOREIGN KEY (wallet_id) REFERENCES wallets(id) ON DELETE CASCADE
);

```